

GitHub

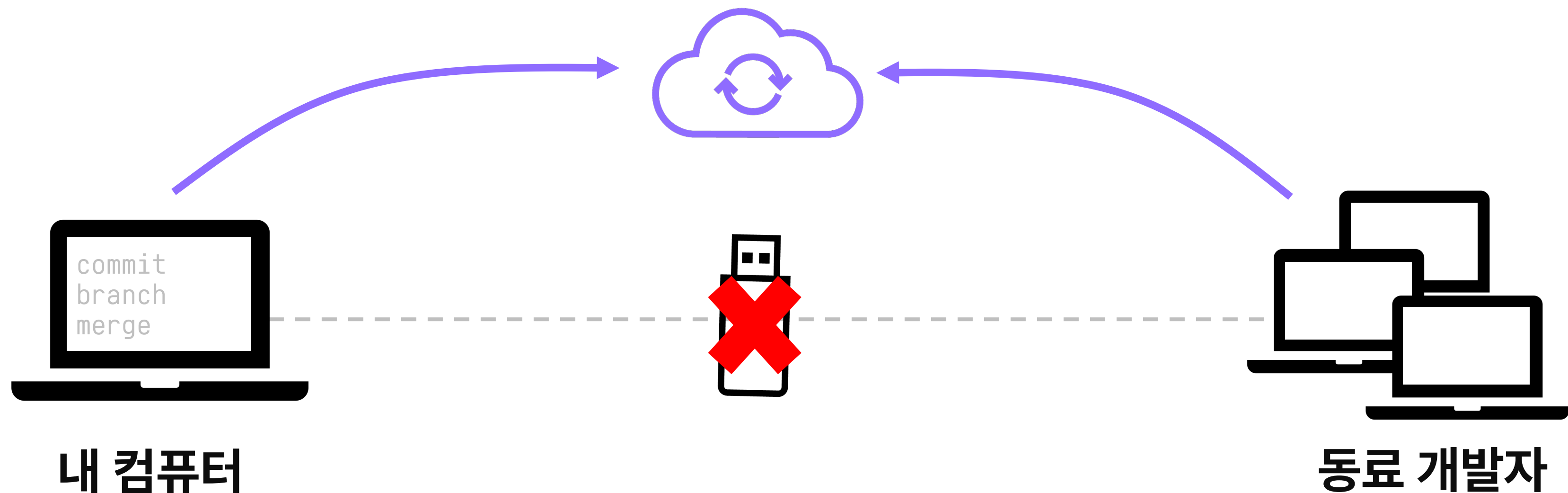
GitHub 협업

GitHub을 활용한 협업 개발 방법

GitHub 협업의 중요성



내 컴퓨터 안에서만 하던 Git, 실무에 간다면?



코드를 한 곳에 모으고, 팀원 간의 충돌을 관리할 **원격 플랫폼**의 필요성



Git의 분산 구조와 협업



분산 버전 관리 시스템

Git은 중앙 서버가 아닌
로컬 저장소 중심으로 작업

모든 사용자가 전체 프로젝트
이력을 보유하여 오프라인
작업이 가능

백업 및 복구 용이

병렬 브랜치 개발 강점



협업의 이점

여러 개발자가 독립된
브랜치에서 동시에 작업 가능

Pull Request로
코드 리뷰 문화 형성

충돌 최소화

커밋 이력 명확히 관리



Git Internals 학습

명령어 작동 원리 이해로
오류 발생 시 빠른 대응

복잡한 커밋 히스토리와
충돌 문제 근본적 이해

고급 기능 사용에
자신감 부여
(rebase, reflog 등)



Git은 '나만의 일기장', GitHub는 '모두의 클라우드'



Git

내 컴퓨터에 설치하는
로컬 버전 관리 도구
(기록, 저장, 가지치기)



GitHub

Git 저장소를 온라인에 올려
팀원들과 함께 쓰는 협업 플랫폼

명확히 구분하는 Git과 GitHub의 역할

구분	Git	GitHub
설치 및 위치	내 컴퓨터 (Local)	✓ 웹 플랫폼 (Cloud)
주요 목적	버전 관리 (Version Control)	✓ 협업 및 공유 (Collaboration)
작업 형태	터미널 명령어 (CLI)	✓ 시각적 웹 인터페이스 (GUI)
핵심 가치	코드를 안전하게 저장하자	✓ 팀원과 코드를 함께 만들자



Branch로 평행하게 작업하기

로그인 UI 개발

프론트엔드 개발자는 사용자가 볼 화면을 만듭니다.

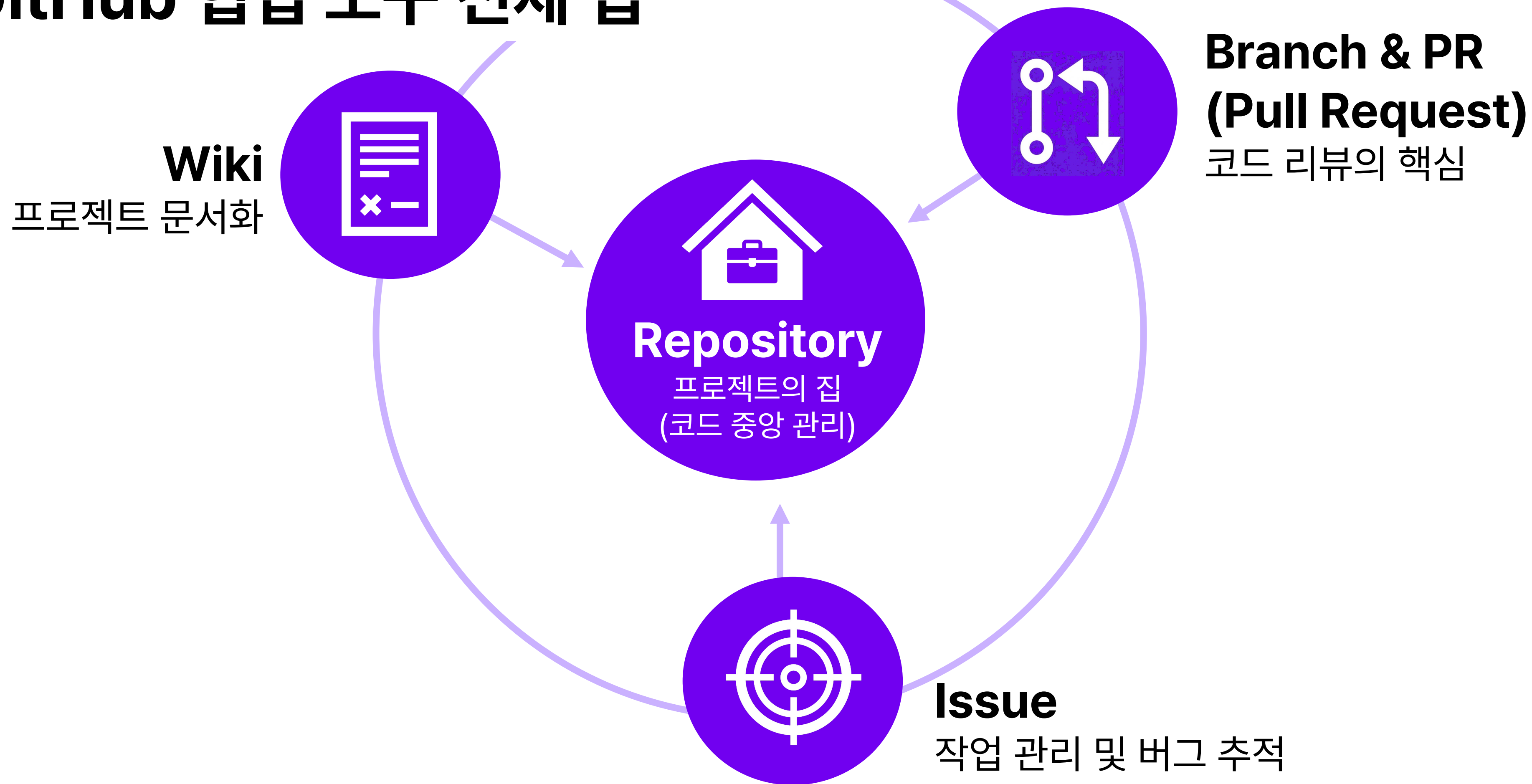
두 사람은 같은 프로젝트 안에서 서로의 작업에 영향을 주지 않고
각자의 Branch에서 평행하게 작업을 진행합니다.

로그인 API 개발

백엔드 개발자는 데이터를 처리할 서버 API를 구축합니다.

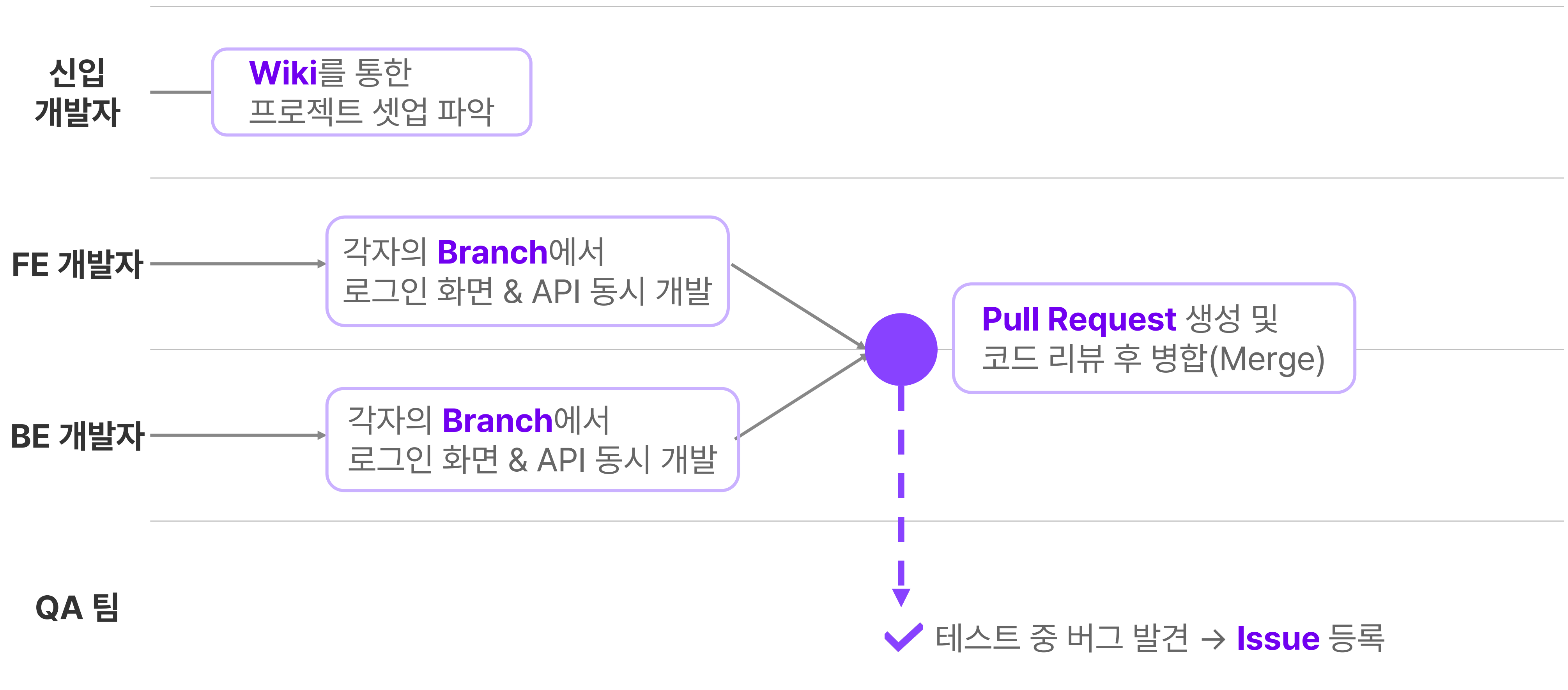


GitHub 협업 도구 전체 맵





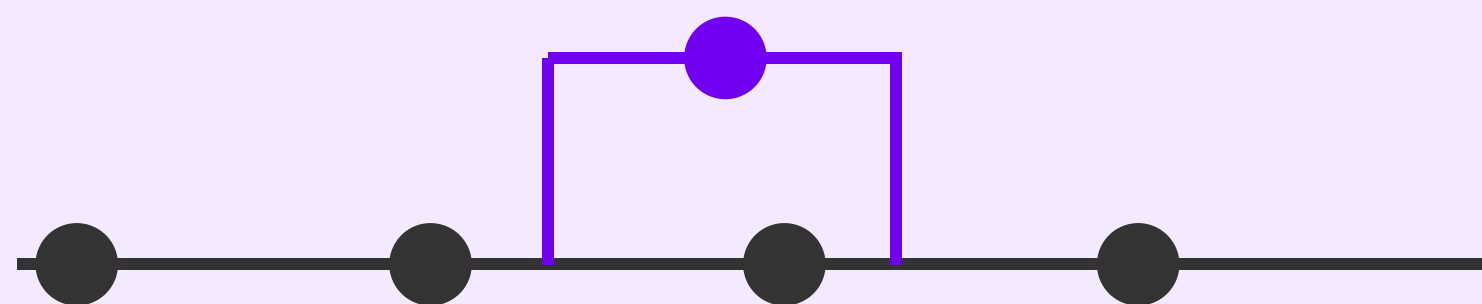
GitHub 협업의 중요성





Repository & Branch

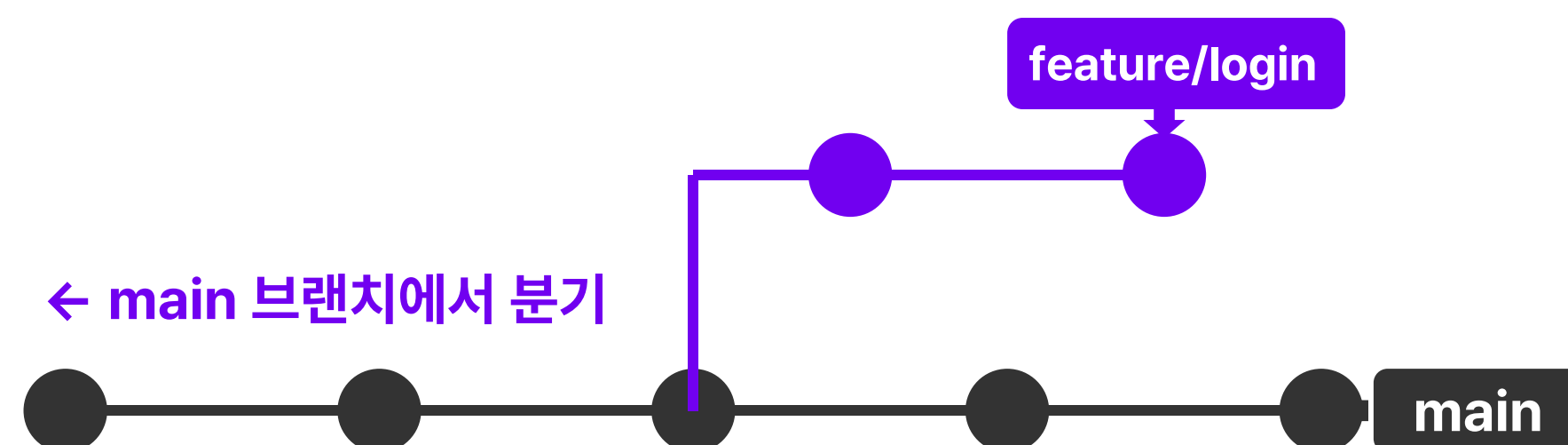
Repository



커밋 · 브랜치 · 히스토리

프로젝트의 모든 것을 담는 물리적 저장소

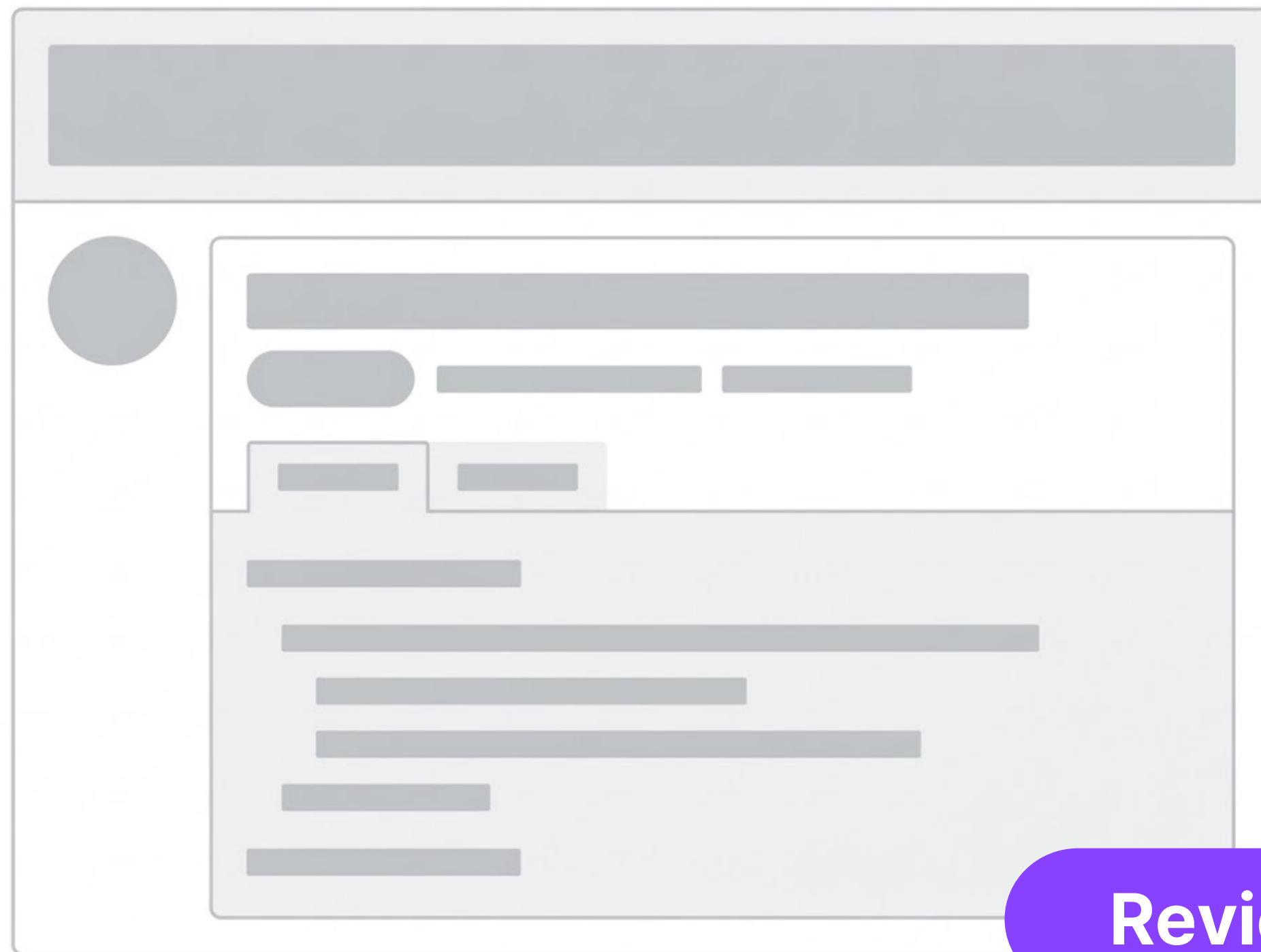
Branch



커밋 그래프에 붙는 논리적 이름표 (포인터)



Pull Request



각자의 Branch에서 완료된 작업을 main Branch에 합치기 전 거치는 필수 관문입니다.

코드 리뷰의 핵심 도구이며, 팀원들에게 내 코드를 설명하고 피드백을 주고받는 가장 중요한 소통의 장입니다.

Review



Issue

할 일	진행 중	완료

'이 기능 개발해야 돼', '여기에 치명적인 버그가 있어'와 같은 모든 논의를 기록하고 추적합니다.

누가, 어떤 문제를, 언제까지 해결할 것인지 명확히 할당하여 프로젝트의 진행 상황을 투명하게 관리합니다.



Wiki

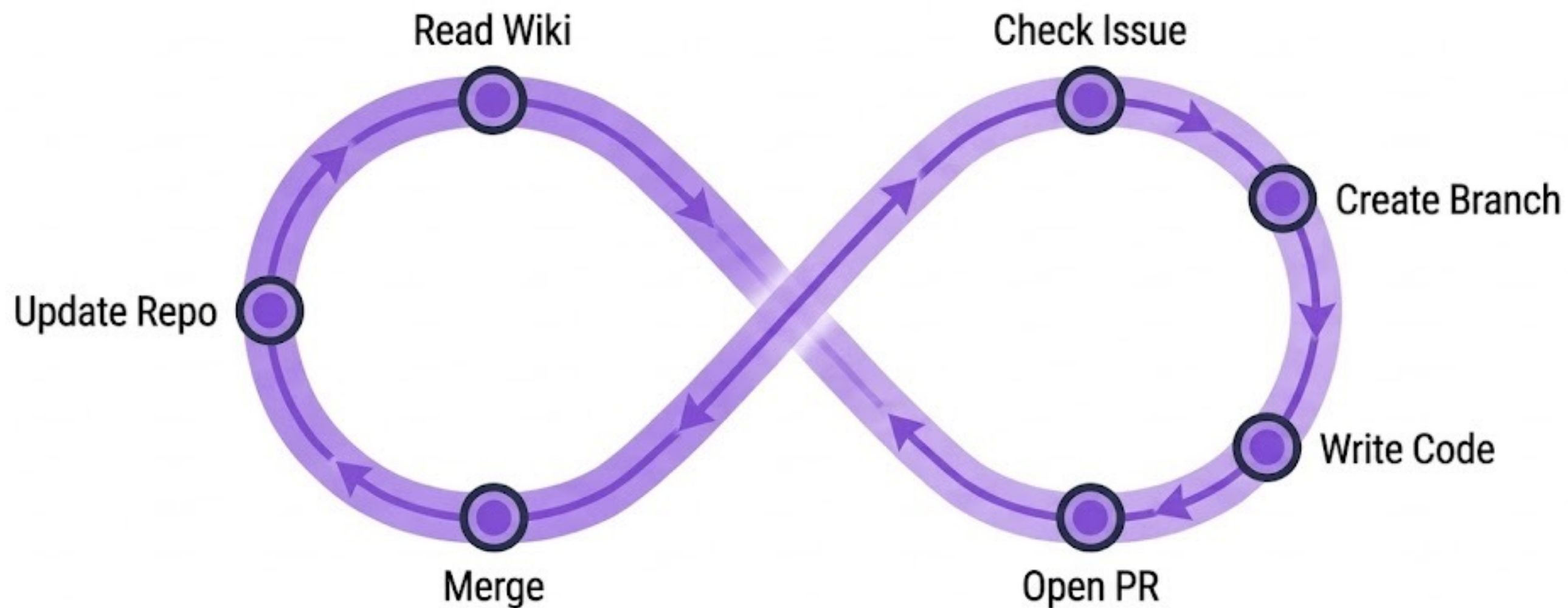


신입 개발자가 들어왔을 때 가장 먼저 확인하는 프로젝트의 설명서입니다.

개발 환경 셋업 가이드, 코딩 컨벤션, 아키텍처 구조 등 누가 와도 바로 프로젝트를 파악할 수 있도록 지식을 자산화합니다.



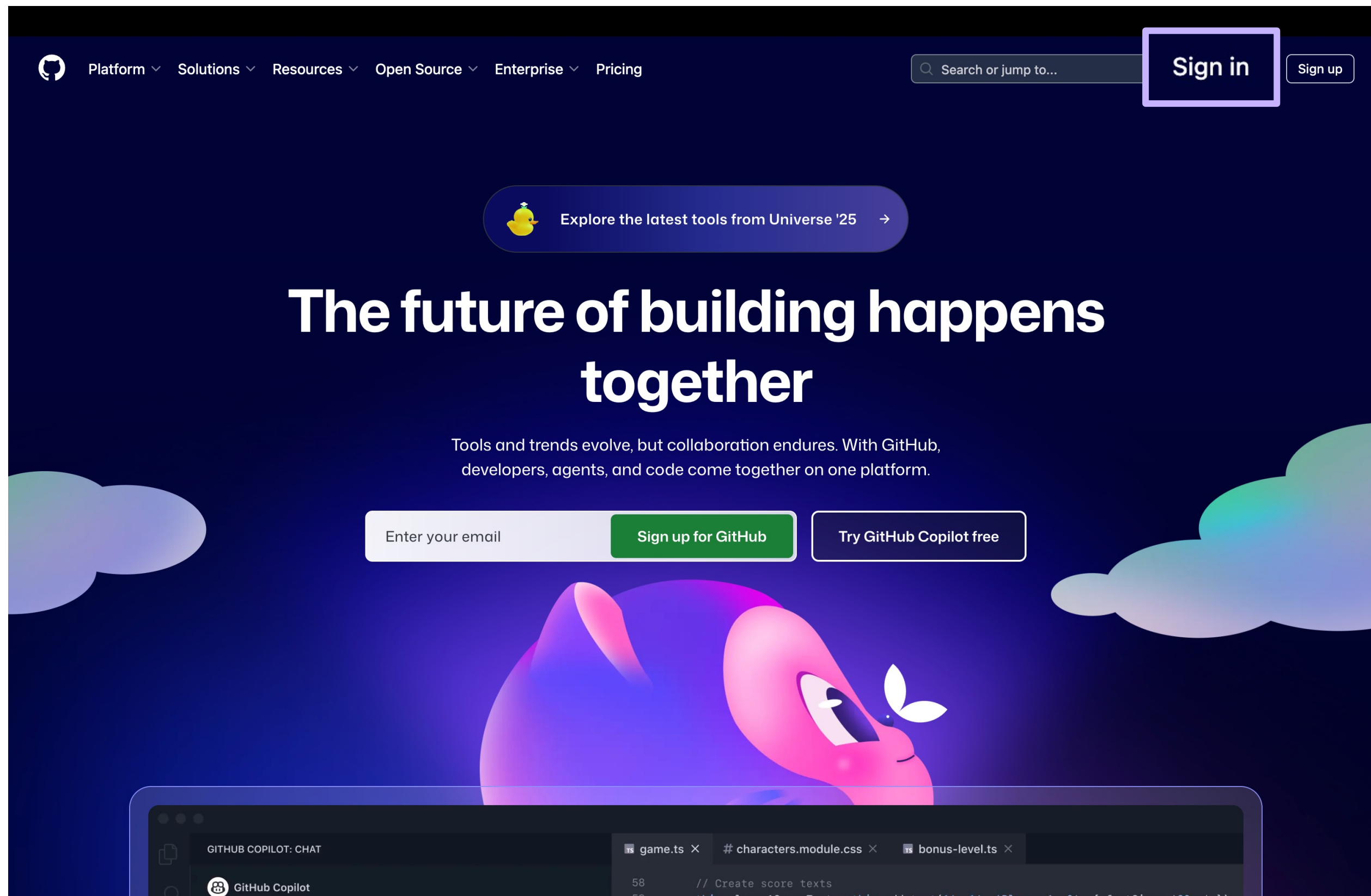
명확히 구분하는 Git과 GitHub의 역할



GitHub의 기능들은 독립적으로 존재하지 않습니다.
가이드를 읽고(Wiki), 할 일을 확인하며(Issue), 코드를 분리해 작성하고(Branch), 리뷰를 거쳐(PR) 최종 완성(Repository)되는 하나의 거대한 사이클입니다.

GitHub 저장소 생성

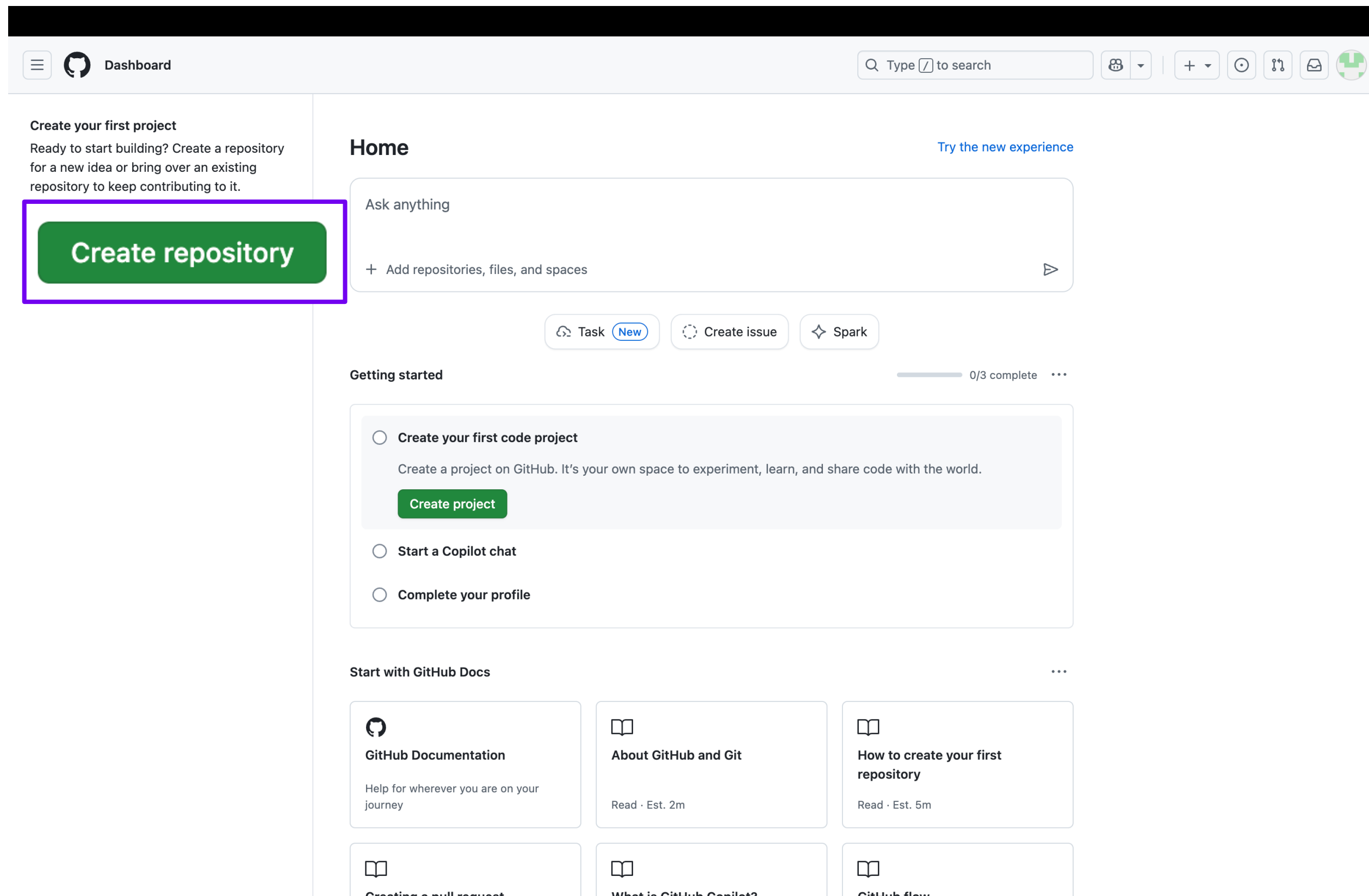
GitHub 접속하기



1. [GitHub](#) 접속하기

2. Sign in 클릭

Create repository



1. Create repository 클릭

레포지토리(Repository)란?

- 하나의 코드 저장소
- 로컬 파일 기준, 하나의 폴더
- 프로젝트 하나 = Repository 하나
- 기술 스택 등에 따라 같은 프로젝트 내 여러 개의 Repository 생성
- 줄여서 '레포(repo)' 라고도 표현

Create repository

New repository

Q Type ↗ to search

1

General

Owner *

yjinnn0719

/

Repository name *

Great repository names are short and memorable. How about **cautious-guacamole**?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

✓ Public

Anyone on the internet can see this repository. You choose who can commit.

Private

You choose who can see and commit to this repository.

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Repository 생성 시 설정해줘야 하는 항목

- Template
- Owner
- Repository name
- Description
- 공개 범위 (Public / Private)
- README file initialize
- .gitignore template
- license

Create repository

New repository

Q Type ↗ to search

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1

General

Owner *

yjinnn0719

Repository name *

Great repository names are short and memorable. How about **cautious-guacamole**?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

1. Repository name 입력

Create repository

New repository

Q Type ↗ to search

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner *

yjinnn0719

Repository name *

Great repository names are short and memorable. How about **cautious-guacamole**?

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off ☐

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

공개 범위가 Public 일 경우

- Repository를 누구나 조회 가능
- 하나의 포트폴리오가 되기도 함
- 외부 서비스 연동 시 비교적 간편
- 민감 정보는 절대 업로드 X (패스워드 등)

공개 범위가 Private 일 경우

- Repository를 특정 사용자만 조회 가능
- 외부 노출이 안되어 비공개 프로젝트 가능
- 무료 사용자에게는 기능이 일부 제한
- 주로 사내 업무는 Private

Create repository

New repository

Q Type ↗ to search

1

General

Owner *

Repository name *

yjinnn0719

/

Great repository names are short and memorable. How about **cautious-guacamole**?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

README?

- Repository에 대해 설명하는 글
- 설치 및 실행 방법, 사용법, 기여하는 방법, License 및 저작권 정보 제공
- 사용자, 개발자가 프로젝트에 대해 이해하기 쉬워짐
- 프로젝트 신뢰도와 완성도가 올라가는 효과
- 원활한 커뮤니케이션을 위한 기본기
- 주로 Markdown 문법으로 작성
- 좋은 README에 대한 참고 링크

<https://github.com/matiassingers/awesome-readme>

Create repository

New repository

Q Type ↗ to search

1

General

Owner *

yjinnn0719

/

Repository name *

Great repository names are short and memorable. How about **cautious-guacamole**?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

.gitignore?

- Git이 무시해야하는 파일이나 폴더를 지정
- 지정되면 Repository에 업로드 되지 않음
- 불필요한 파일, 비밀번호 등 보안 파일을 지정하여 효율적이고 보안을 유지한 운용
- 다양한 언어와 프레임워크 별 템플릿 존재

Create repository

New repository

Q Type ↗ to search

1

General

Owner *

Repository name *

yjinnn0719

/

Great repository names are short and memorable. How about **cautious-guacamole**?

Description

0 / 350 characters

2

Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

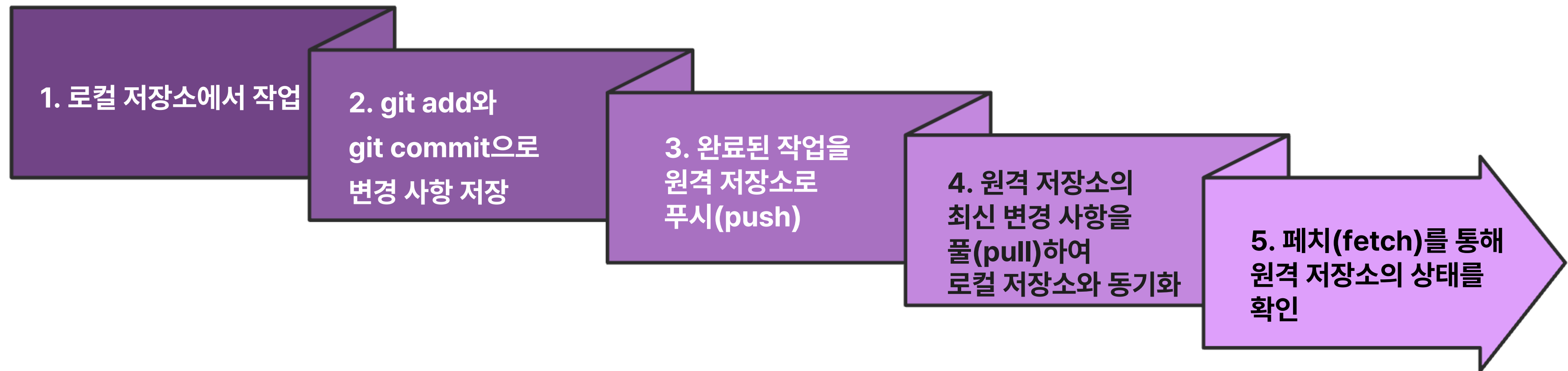
license?

- SW 사용, 배포, 수정 등의 권한을 정의
- 사람들이 어떤 조건으로 코드를 사용하고 변경할 수 있는지를 명확히 규정
- 각 license 마다 사용과 배포 조건이 다름
- 개인 공부나 간단한 프로젝트에는 지정하지 않아도 무방
- 타인의 코드 사용 시 반드시 확인해야 함
- 관련 사이트

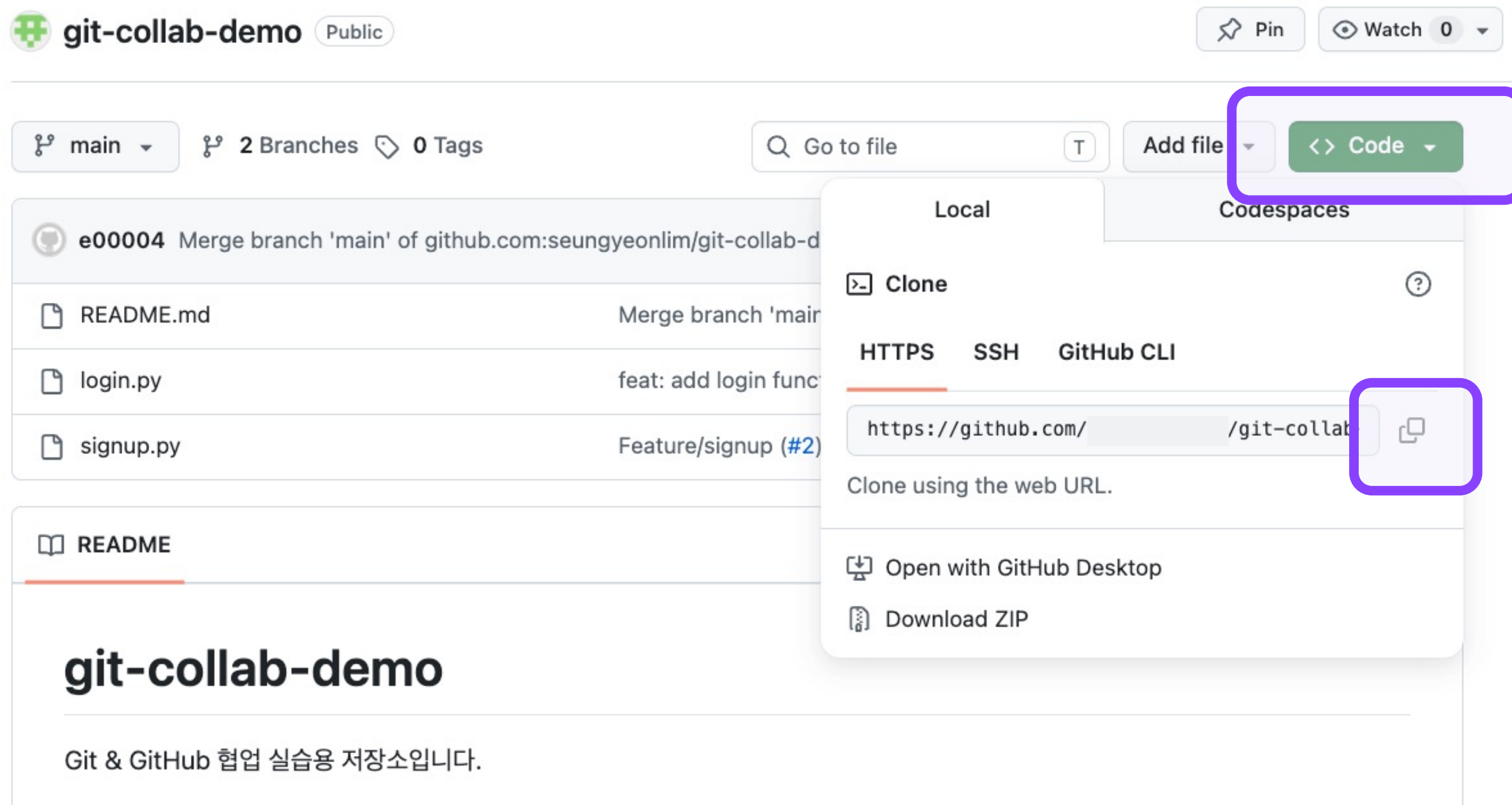
<https://choosealicense.com/>



로컬 저장소와 원격 저장소는 **동기화**를 통해 상호 작용



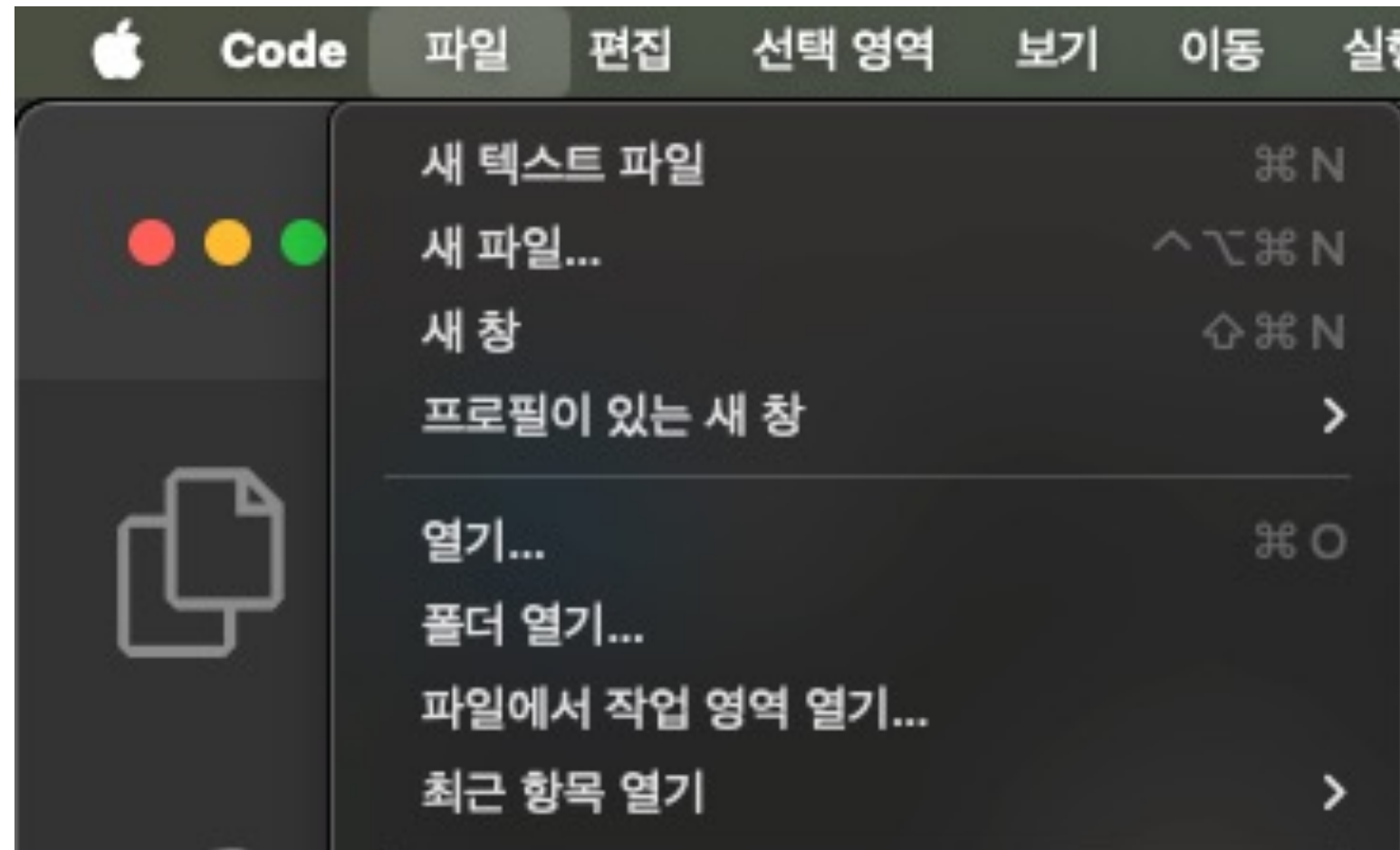
Clone repository



1. GitHub Repository의
Code 버튼 클릭

2. **HTTPS 옵션**의
주소를 복사

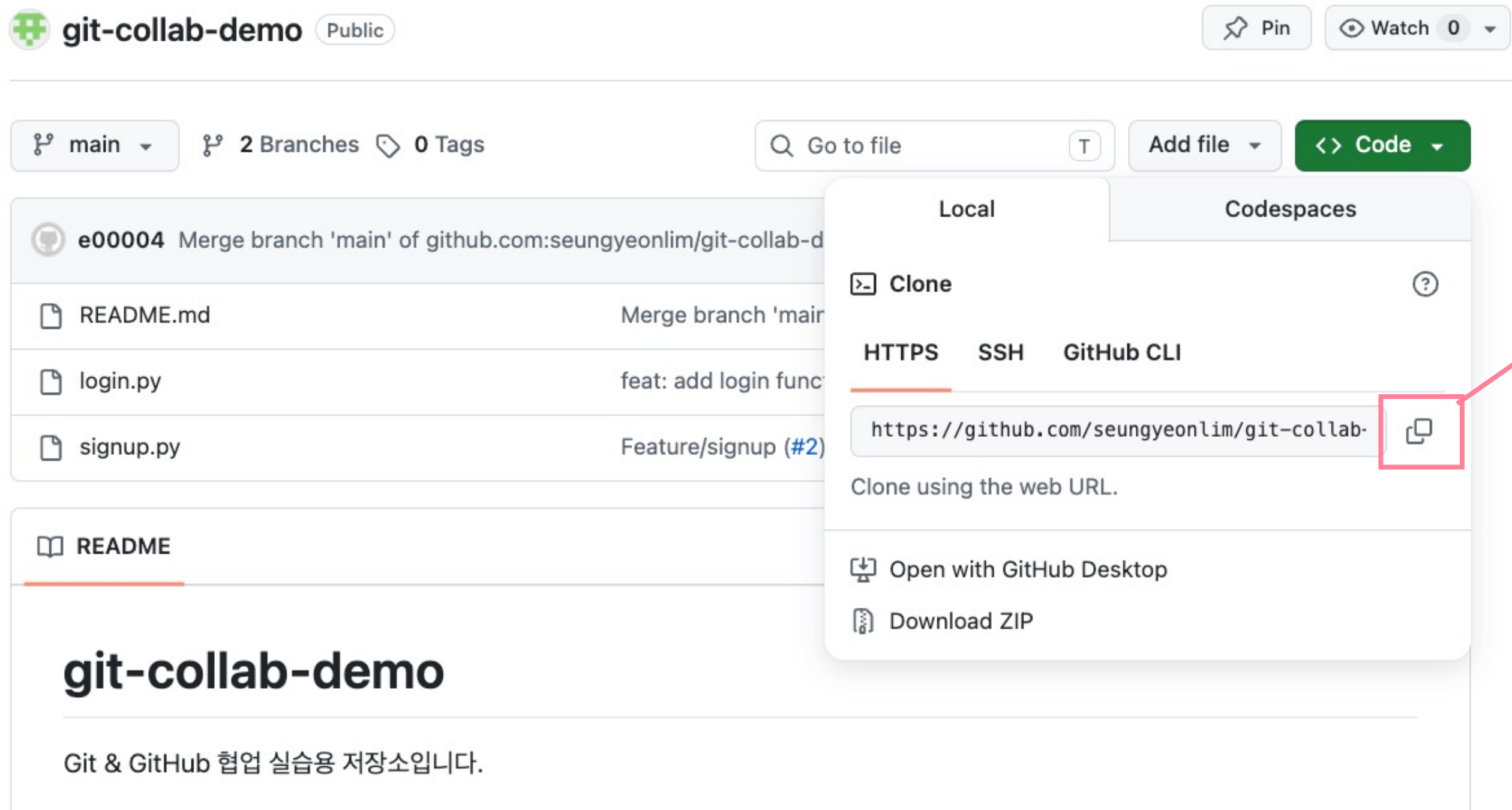
Clone repository



```
(base) git-test %
```

해당 repository를 받아올 새로운 디렉토리 생성 및 IDE에서 열기

Clone repository



The screenshot shows the GitHub interface for a repository named 'git-collab-demo'. The repository is public and has 2 branches and 0 tags. The 'Code' button is highlighted, and the 'Clone' dropdown menu is open. The 'Clone' menu shows the repository URL: `https://github.com/seungyeonlim/git-collab-`. The 'Clone' button is highlighted with a red box. A red arrow points from this box to the `<repository_url>` placeholder in the terminal command shown in the adjacent block.

```
$ git clone <repository_url>
```

원격 저장소에 있는
git repository를
복사하는 명령어



반대로 현재 작업하던 폴더를 Repository에 연결하려면?

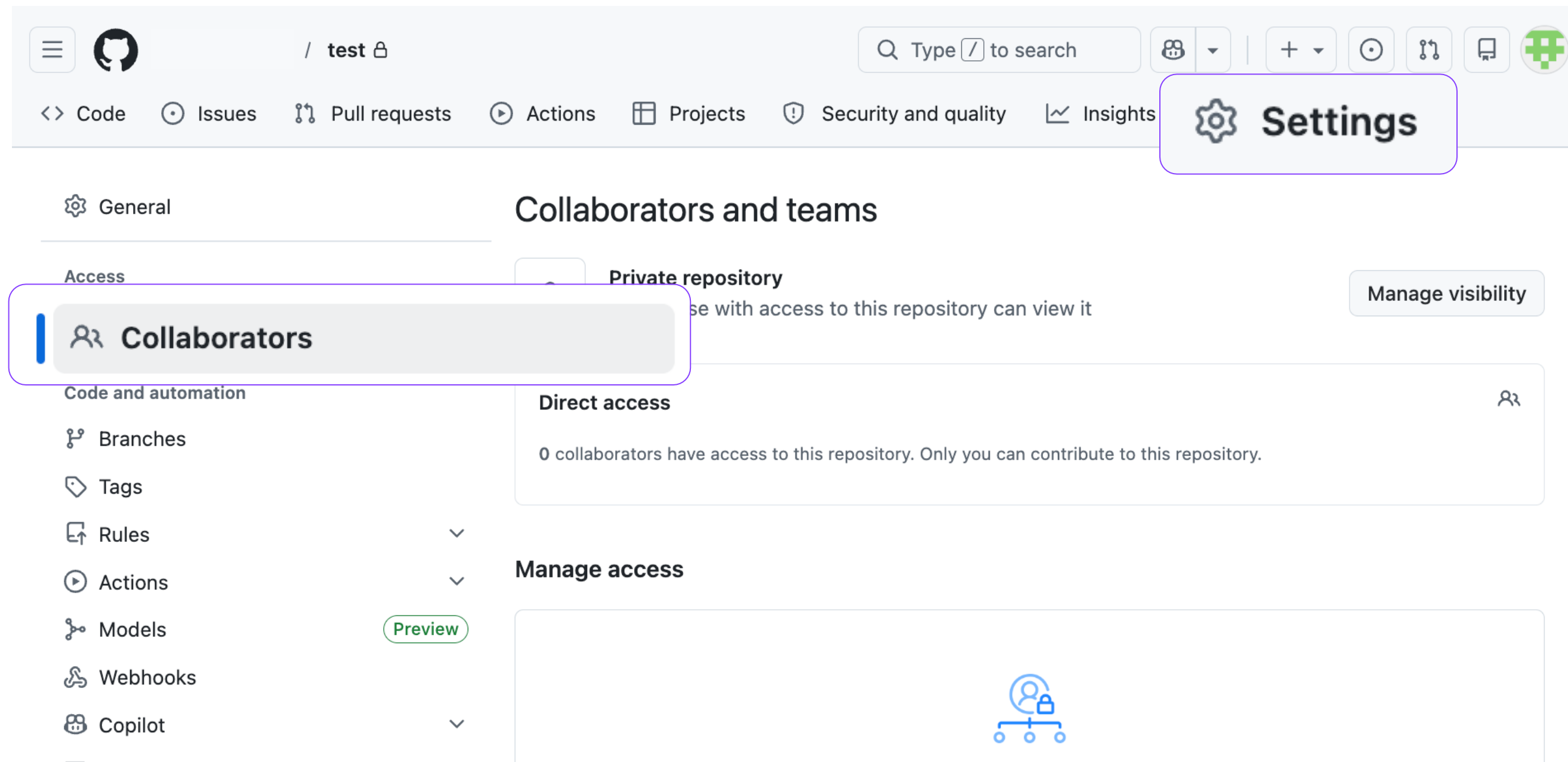
```
echo "# test" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin https://github.com/elice/test.git
git push -u origin main
```

내 컴퓨터에 저장되어 있는 저장소와
원격 저장소를 연결하기 위해서 사용하는 명령어

원격 저장소의 단축 이름을
origin으로 지정한다는 의미

효율적인 협업을 위한 Collaborator

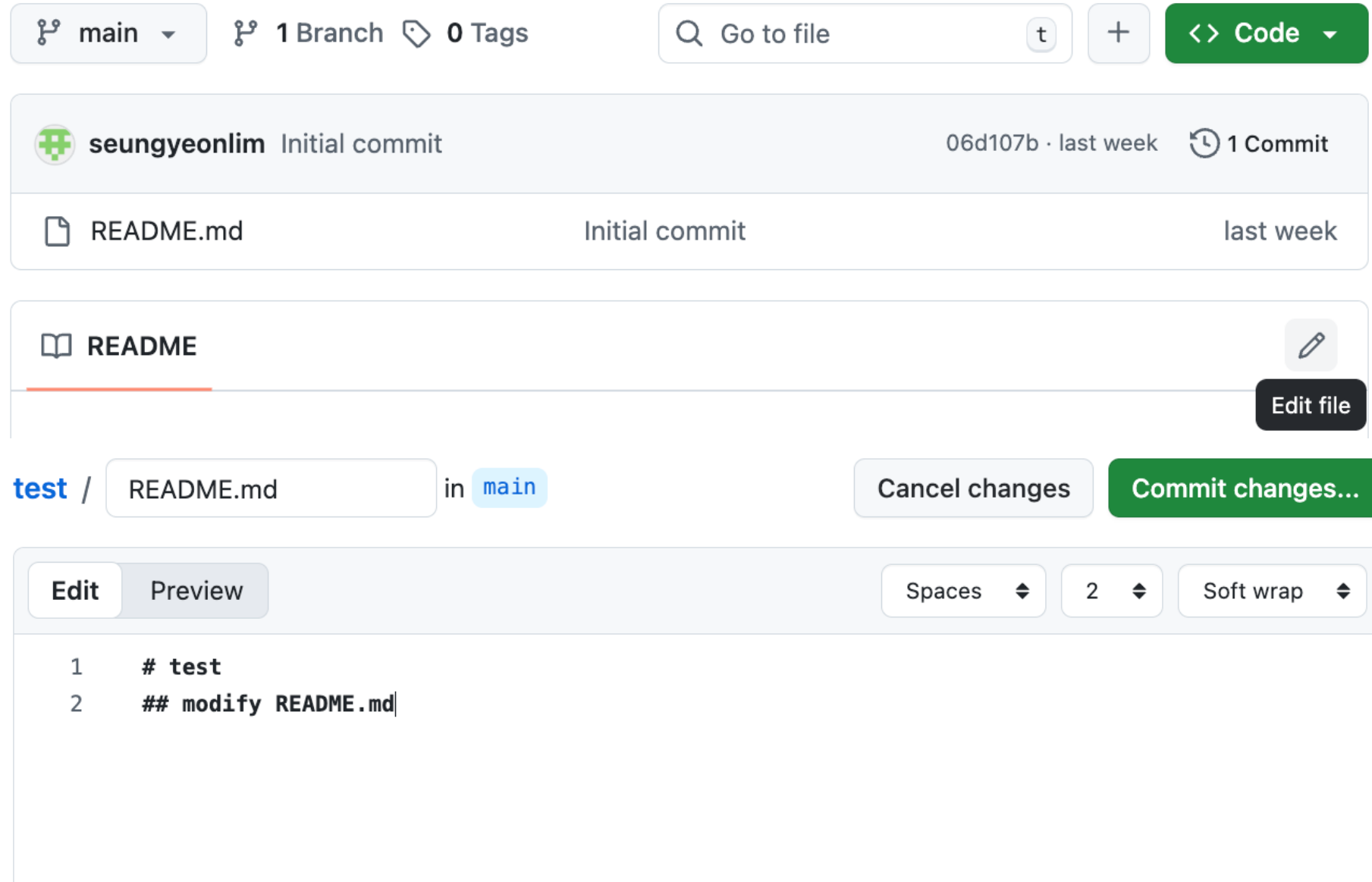
효율적인 협업을 위한 Collaborator



Collaborator(팀원) 등록방법

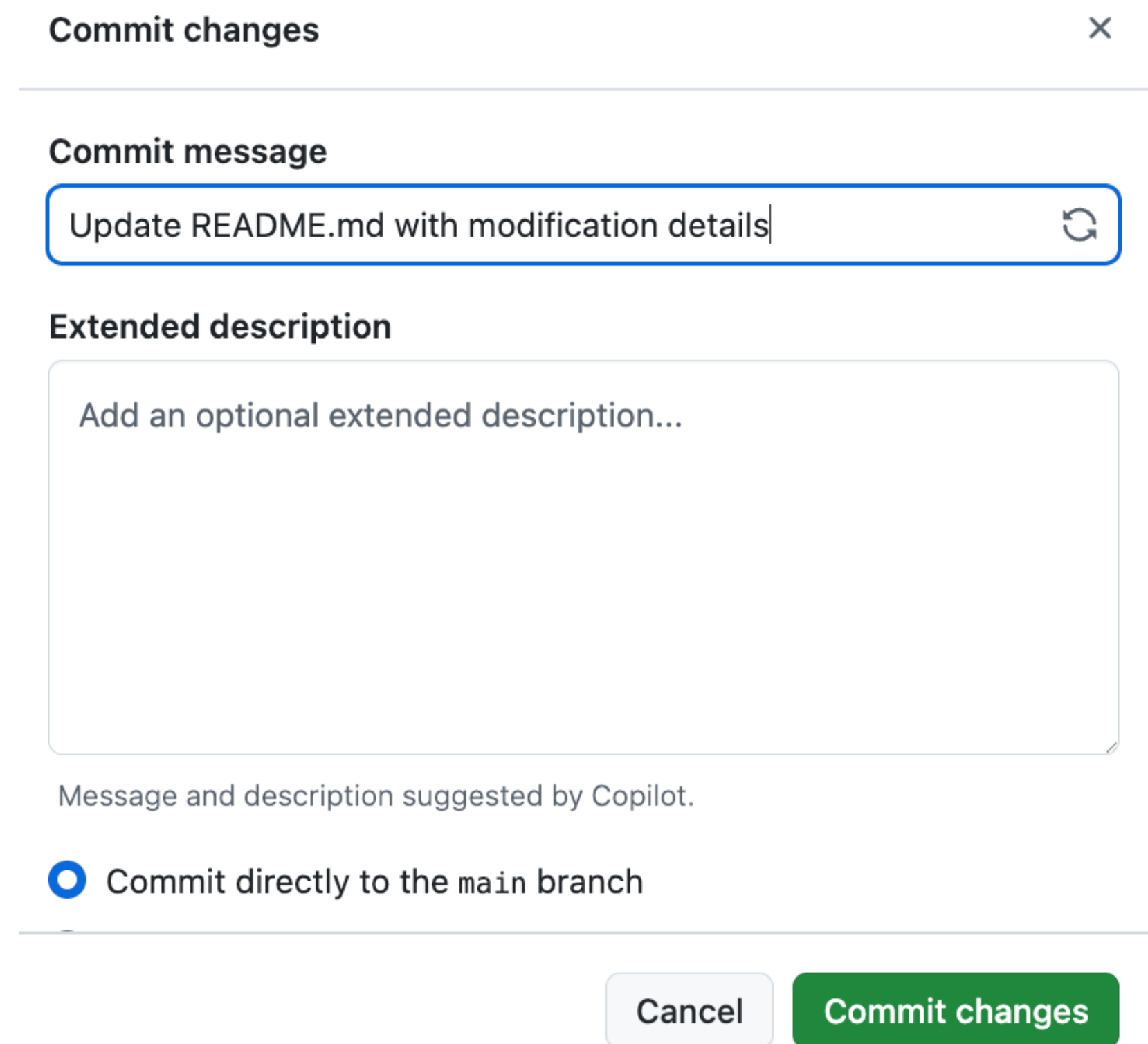
- Repository ⇒ Settings
⇒ Collaborators에서 팀원 초대
- 초대 받은 사람이 수락을 해야 함

효율적인 협업을 위한 Collaborator

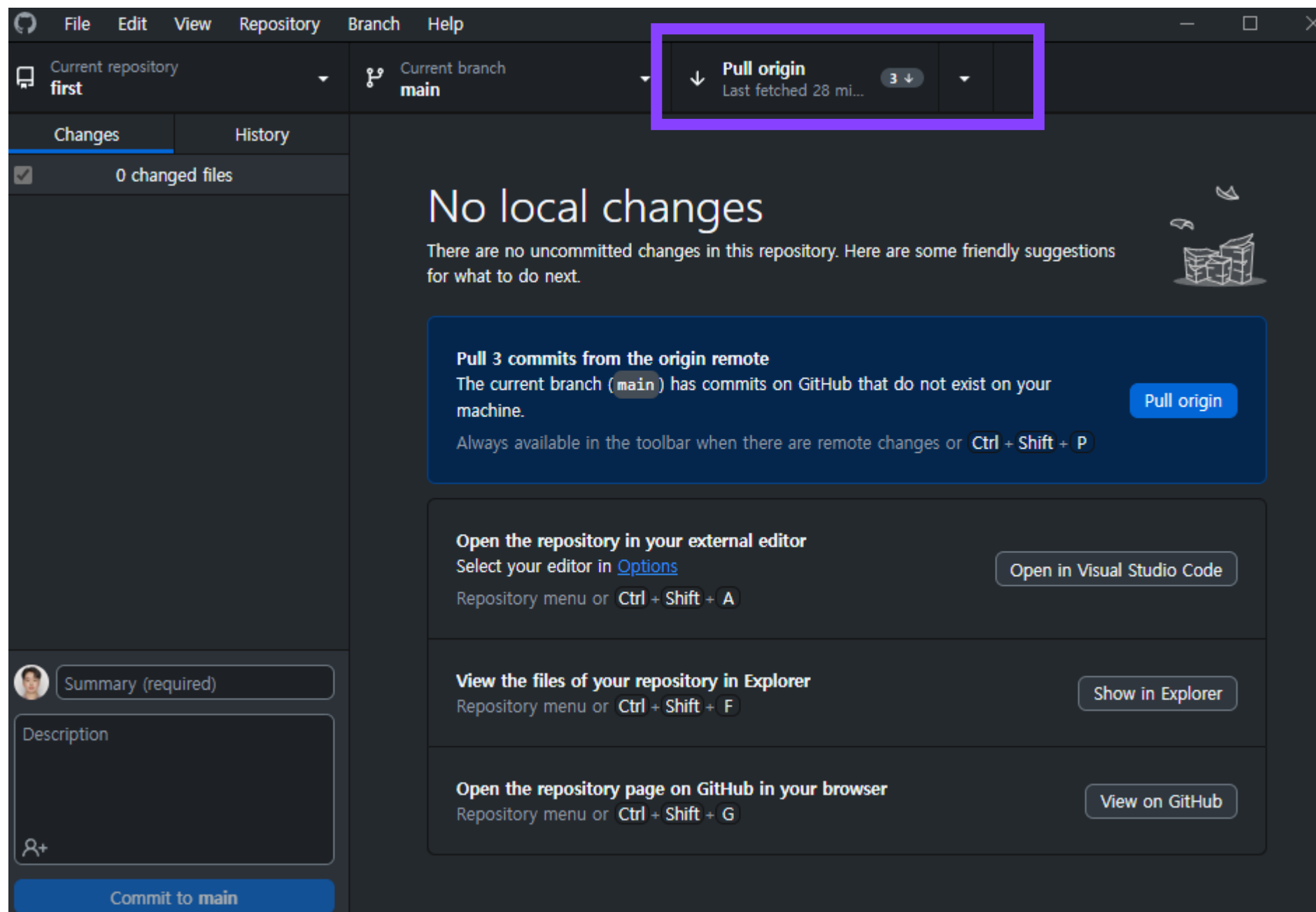


Collaborator 등록 시

- Repository를 직접 수정 가능 (연필 모양 버튼)
- 웹 상에서 바로 commit, push 가능

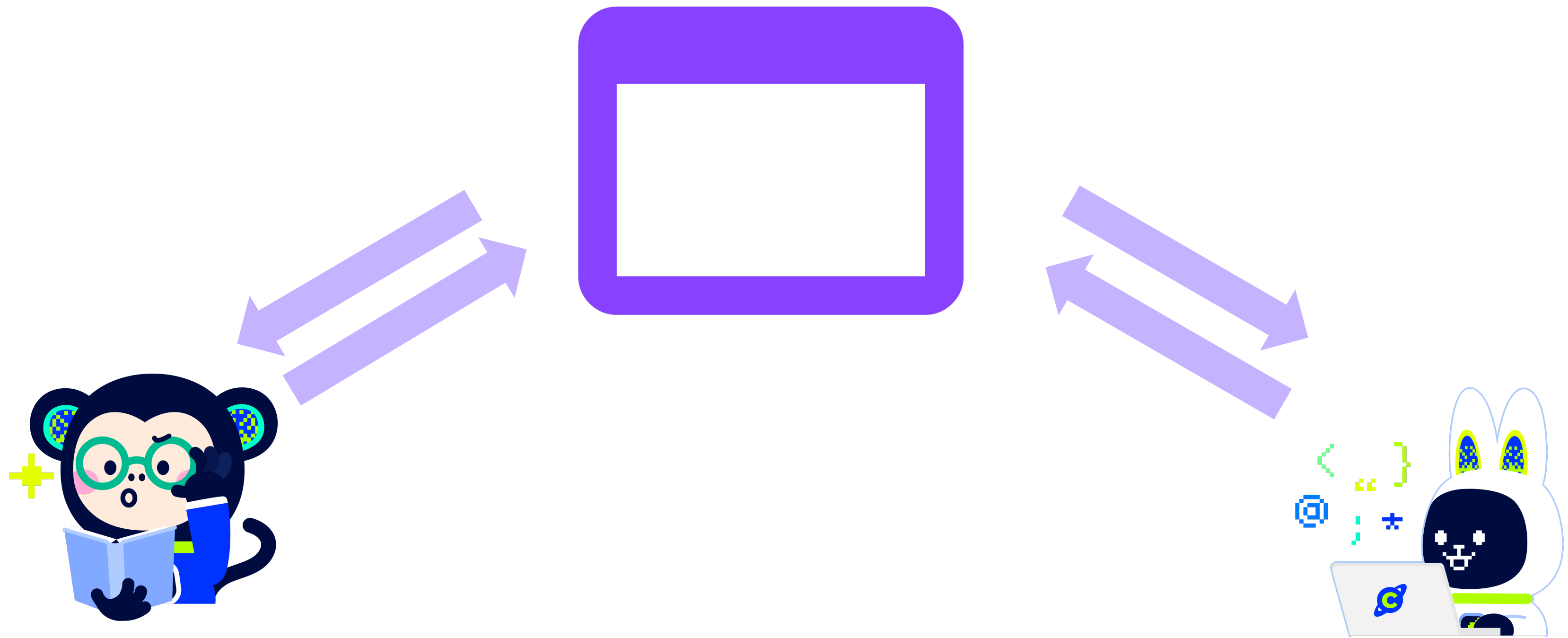


효율적인 협업을 위한 Collaborator



최신 버전 유지를 위해서,
Remote Repository의 변경 사항 로컬 적용 필수
→ Pull 필수

최신 버전이 유지되지 않을 때,
충돌이 발생하여 원활한 협업 불가능



같은 Remote Repository를 직접 수정

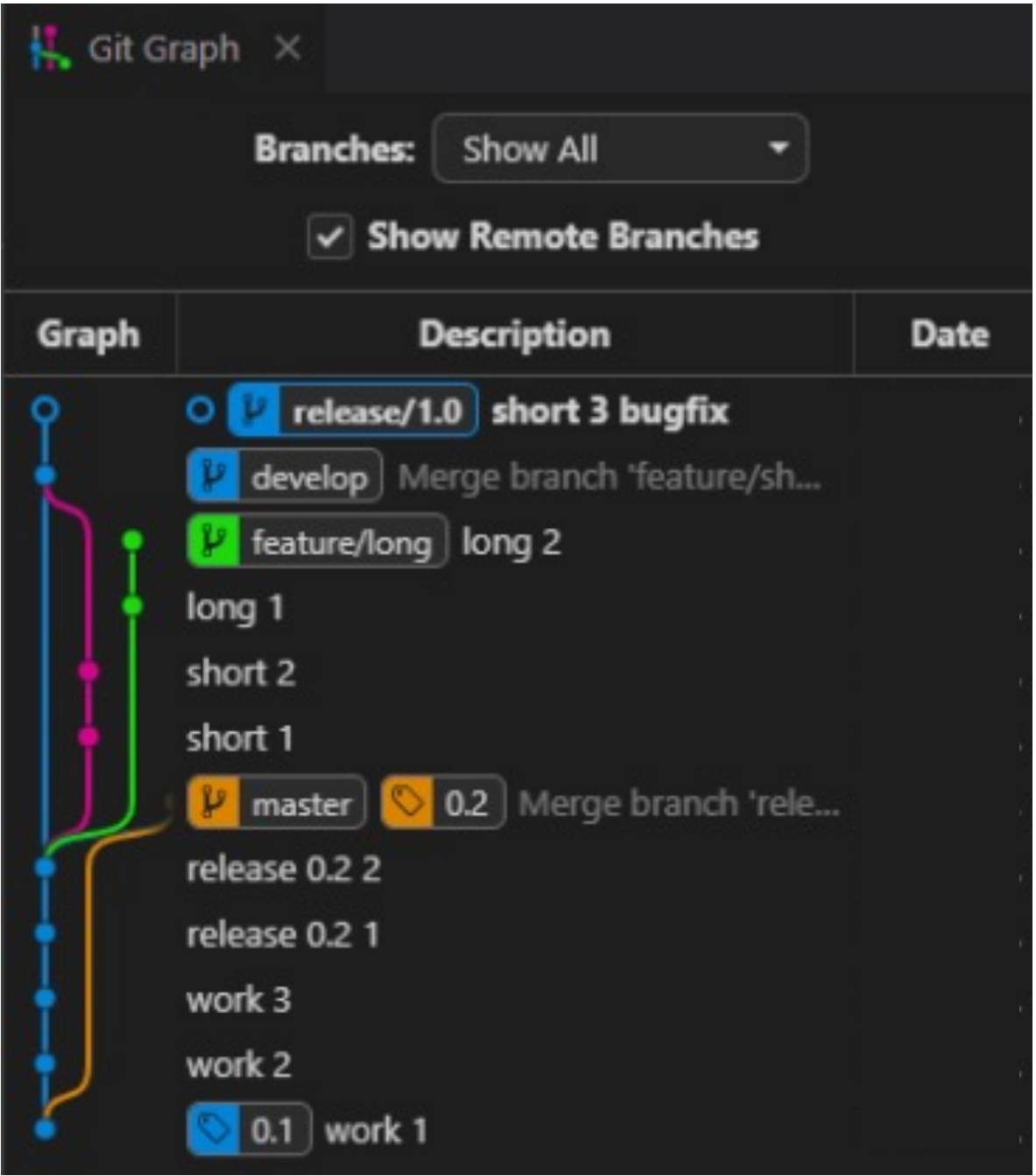
Branch 전략과 Merge

Git Flow 설계 방법

- 중대형 프로젝트에 적합한 전략으로 병렬 개발을 위한 전략으로 주로 사용이 됨
- 해당 전략은 'Vincent Driessen'이 2010년도에 제안한 방식으로 기능 개발과 유지보수를 위한 브랜치 전략을 제공함

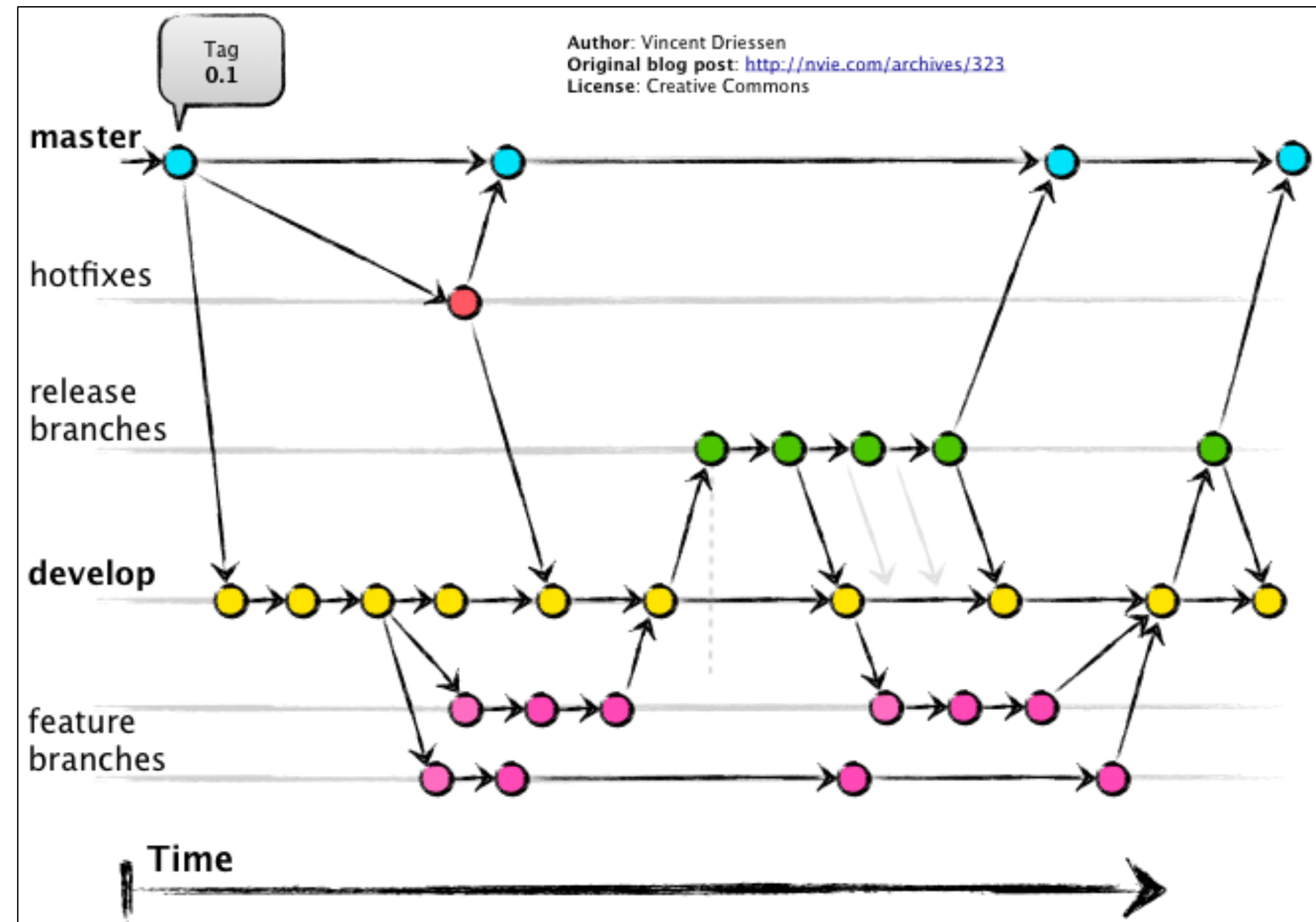
주요 브랜치

브랜치 이름	설명
main(master)	제품 출시 버전을 관리하는 브랜치
develop	다음 출시 버전을 개발하는 브랜치
feature	새로운 기능을 개발할 때 사용하는 브랜치
release	다음 출시 버전을 준비하는 브랜치
hotfix	긴급한 버그 수정이 필요할 때 사용하는 브랜치



Git Flow 브랜치의 흐름

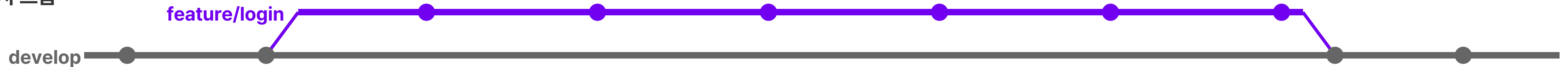
단계	브랜치	핵심 역할	흐름
1	develop	개발 최신 코드	시작
2	feature	기능 개발	develop → feature
3	feature 병합	기능 완료	feature → develop
4	release	릴리즈 준비	develop → release
5	release 병합	안정화	release → develop
6	main 배포	최종 릴리즈	develop → main
7	main	안정 코드	-
8	hotfix	긴급 수정	main → hotfix → main



실무 워크플로우 ① Feature 개발 흐름



브랜치 흐름





Branch 생성과 Pull Request

실무 워크플로우 ② Release & Hotfix 흐름

Release · 계획된 배포

1. release 분기

```
git checkout -b  
release/1.2.0  
develop
```

develop에서 컷

2. 베타 테스트

QA / 스테이징
배포 검증

기능 동결

3. 버그 픽스

```
git commit -m  
"fix: ..."
```

release에서만 수정

4. main 머지 + 태그

```
git merge  
release/1.2.0  
git tag v1.2.0
```

운영 반영

5. develop 백머지

```
git checkout develop  
git merge  
release/1.2.0
```

수정사항 동기화

Hotfix · 긴급 운영 패치

1. 운영 버그

Sentry / 고객 리포트

main에서 출발

2. hotfix 분기

```
git checkout -b  
hotfix/login-crash  
main
```

develop이 아닌 main

3. 수정 + 검증

```
git commit -m  
"fix: login crash"
```

최소 변경

4. main 머지 + 태그

```
git merge  
hotfix/login-crash  
git tag v1.2.1
```

즉시 배포

5. develop 백머지

```
git checkout develop  
git merge  
hotfix/login-crash
```

동일 버그 재발 방지



브랜치 네이밍 가이드(1/2)

- 효율적인 협업과 코드 관리의 일관성을 위해 브랜치 네이밍 규칙을 지켜야 함
- 브랜치 이름만 보더라도 목적, 작업 범위, 관련 이슈를 쉽게 파악할 수 있도록 작성함

1. 기본 규칙

- 소문자와 하이픈(-)만 사용함
- 공백과 특수문자는 사용하지 않음
- 브랜치 이름은 짧고 명확하게 작성함

2. 브랜치 유형 및 접두사(prefix)

유형	접두사 예시	설명
기능 개발	feature/	새로운 기능 개발 시 사용
버그 수정	bugfix/	일반 버그 수정
긴급 수정	hotfix/	프로덕션 환경의 긴급 버그/보안 수정
릴리즈 준비	release/	릴리즈 버전 준비 및 QA
실험 / 테스트	experiment/	실험적 기능 개발
문서 작업	docs/	문서 수정 및 추가

```
~/test-project on main
> git flow init

Which branch should be used for bringing forth production releases?
- main
Branch name for production releases: [main]
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? []
Hooks and filters directory? [/Users/undo/test-project/.git/hooks]
```



브랜치 네이밍 가이드(2/2)

3. 네이밍 패턴

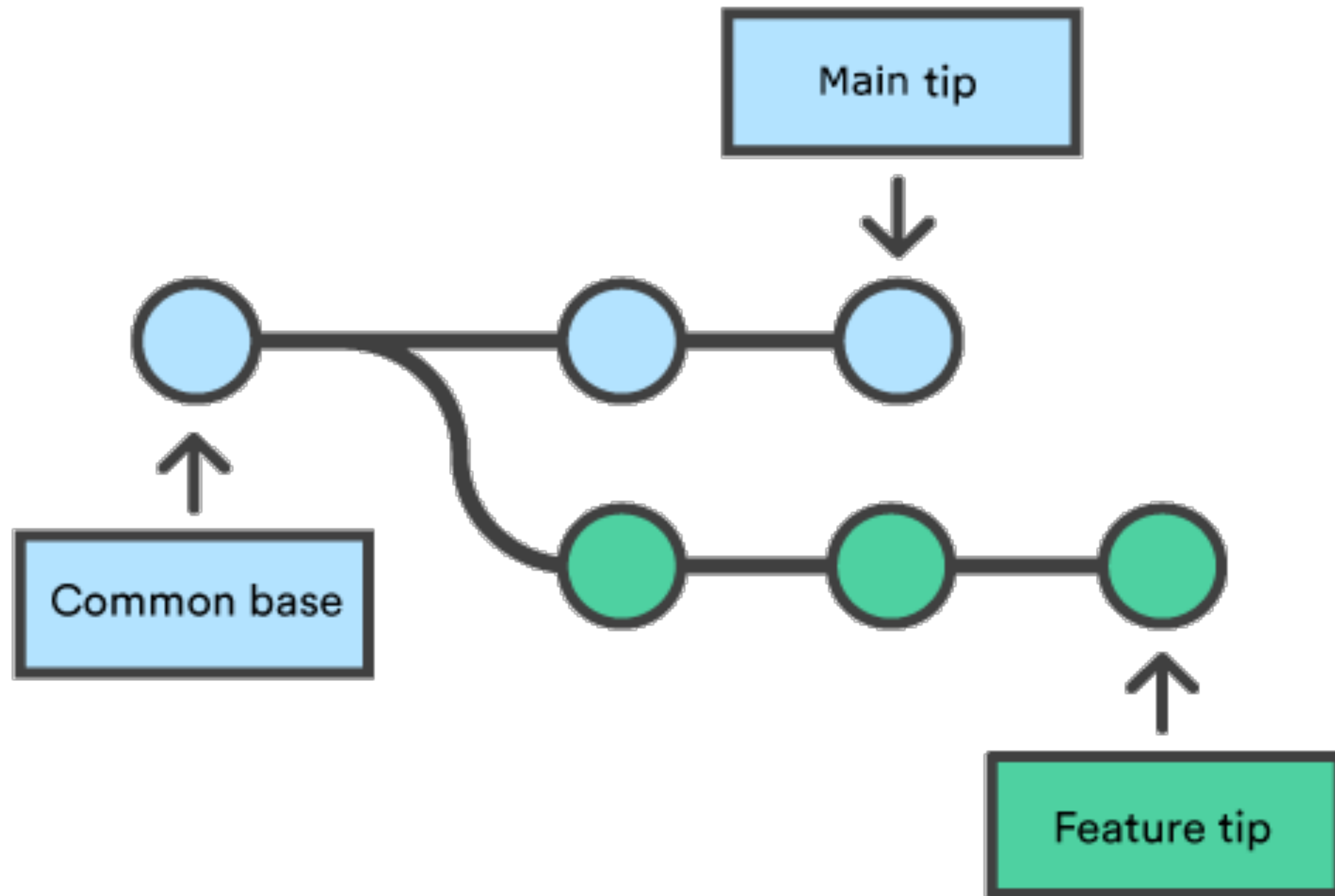
- type : 브랜치 유형 접두사(feature, bugfix, hotfix, ...)
- short-description : 작업 내용을 간결하게 설명 (하이픈으로 구분)
- Issue-number : (선택) GitHub/Jira 등의 이슈 번호

4. 예시

로그인 기능 추가	feature/login-api-#45
장바구니 수량 오류 수정	bugfix/cart-quantity-#102
서버 응답 지연 긴급 수정	hotfix/server-latency-#210
1.2.0 버전 릴리즈 준비	release/1.2.0
API 문서 업데이트	docs/api-update-#88



병합(Merge)



- 한 브랜치의 변경 사항을 다른 브랜치로 통합하는 과정을 의미함
- Git에서는 두 개의 브랜치를 하나로 합치는 작업을 수행할 때 이를 이용함
- 이러한 병합 과정을 통해서 프로젝트의 모든 구성원이 동일한 메인 코드베이스에서 작업할 수 있음

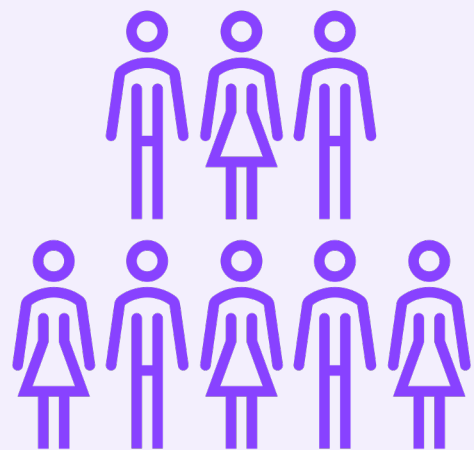


왜 브랜치 병합 전략이 필요할까?

동시 작업의 효율성 향상

독립 브랜치에서
개발/커밋 가능

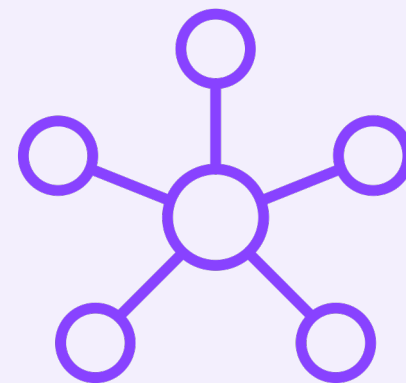
충돌은 Merge
시점에서만 해결



명확한 목적을 가진 브랜치 구조

환경별 브랜치 분리

main(배포),
develop(테스트),
feature/login(기능)



버전 관리 및 롤백의 명확성

안정 브랜치 명확 →
빠른 롤백

비상 상황에서도
신속 대응 가능





브랜치 병합 전략(Branch Merge Strategy)(2/3)

- GitHub에서 브랜치들을 메인 브랜치로 병합할 때 사용할 수 있는 규칙들을 의미함
- 각각의 전략은 서로 다른 장단점을 가지고 있어 프로젝트의 특성과 팀의 워크플로우에 따라 적절한 규칙을 선택해야 함
- 특히, 프로젝트의 소스코드 히스토리를 어떻게 관리할 것인지에 대한 팀의 합의가 중요함

병합 전략	설명	커밋 히스토리	단점
Merge Commit (기본 병합)	모든 커밋 히스토리를 보존하면서 병합하는 방식	유지됨	히스토리가 복잡해짐
Squash and Merge	여러 커밋을 하나의 커밋으로 압축하여 병합하는 방식	압축됨	작업 세부 기록은 사라짐
Rebase and Merge	커밋들을 베이스 브랜치 위로 재배치하는 선형적 방식	변형됨	충돌 발생 시 복잡, 커밋 해시 변경



브랜치 병합 전략(Branch Merge Strategy)(3/3)

- 병합 전략 설정
- 기본적으로 merge commit 방식으로 설정되어 있음
- 추가적인 병합 방법 설정이 필요하면 아래의 과정을 통해서 설정 가능함
- Github → Settings → General → Pull Requests 순서로 지정

Pull Requests

When merging pull requests, you can allow any combination of merge commits, squashing, or rebasing. At least one option must be enabled. If you have linear history requirement enabled on any protected branch, you must enable squashing or rebasing.

☒ **Allow merge commits**
Add all commits from the head branch to the base branch with a merge commit.
Default commit message
Presented when merging a pull request with merge.
Default message ▾

☒ **Allow squash merging**
Combine all commits from the head branch into a single commit in the base branch.
Default commit message
Presented when merging a pull request with squash.
Default message ▾

☒ **Allow rebase merging**
Add all commits from the head branch onto the base branch individually.

Branch 생성과 Pull Request



Branch 생성과 Pull Request

Branch 생성하기

```
bash - ~/projects/my-repo

$ git branch
* main

# 새 브랜치 만들고 바로 이동 (Git 2.23+ 권장)
$ git switch -c feature/readme
Switched to a new branch 'feature/readme'

# 또는 (기존 방식): git checkout -b feature/readme
$ git branch
main      * feature/readme
```

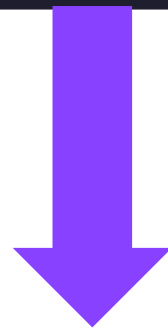
- 1 프로젝트 디렉토리로 이동**
터미널에서 `cd` 명령으로 작업할 레포지토리 폴더로 이동합니다.
- 2 새 브랜치 생성 + 즉시 전환**
`git switch -c <브랜치명>` 을 실행하면 현재 브랜치를 기준으로 새 브랜치가 생성되고 동시에 그 브랜치로 이동합니다.
- 3 git branch 로 결과 확인**
`git branch` 를 다시 실행해 브랜치 목록과 * 표시(현재 위치)를 확인합니다.

switch -c vs checkout -b

`git switch -c` 는 Git 2.23(2019) 도입 전용 명령(권장), `git checkout -b` 는 동작이 같은 기존 명령입니다.

README 편집하기

```
bash
# 에디터로 파일 열기
$ code README.md
```



```
i README.md ●
i README.md > abc # First Repository
1 # First Repository
2 **Git & GitHub 학습용 저장소**입니다.
3 - 브랜치, Pull Request, merge 등 협업 기능을 실습합니다.
```

1 README.md 파일 열기

터미널에서 `code README.md` 를 실행하거나, VSCode에서 직접 파일을 클릭해 엽니다.

2 내용 수정 후 저장 (Ctrl + S)

Markdown 문법으로 제목·강조·목록을 작성합니다.
수정이 끝나면 `Ctrl + S` (Mac은 `⌘ + S`) 로 저장하세요.

Markdown 빠른 문법

`# 제목` · `**굵게**` · `- 목록` · `[링크](url)`
에디터는 VSCode가 아니어도 메모장·Sublime·Vim 등 어떤 편집기로 열어 저장해도 같은 결과입니다.



Branch 생성과 Pull Request

변경사항 확인 & Commit

```
bash — feature/readme

$ git status
On branch feature/readme
Changes not staged for commit:
  modified:   README.md

# 변경된 내용 라인 단위로 확인
$ git diff
- # Project
+ # My Project

# 변경사항 스테이징 후 커밋
$ git add README.md
$ git commit -m "docs: update README title"
[feature/readme a3f9c21] docs: update README title
1 file changed, 1 insertion(+), 1 deletion(-)
```

- 1 git status — 현재 상태 확인**
어떤 파일이 **변경(modified)** 또는 새로 추가(**untracked**)되었는지 보여줍니다.
현재 브랜치명도 함께 표시되어 작업 위치를 확인할 수 있습니다.
- 2 git diff — 라인 단위로 변경 내용 보기**
이전 커밋과 비교해 어떤 줄이 바뀌었는지 보여줍니다.
- 빨간 줄 = 삭제 / + 초록 줄 = 추가
- 3 git add → git commit — 변경 확정**
`git add <파일>` 로 스테이징하고, `git commit -m "..."` 로 메시지와 함께 기록합니다.
커밋은 로컬 브랜치에만 저장되며, 원격(GitHub)에는 아직 반영되지 않습니다.



Branch 생성과 Pull Request

원격 저장소에 PUSH

```
bash — feature/readme

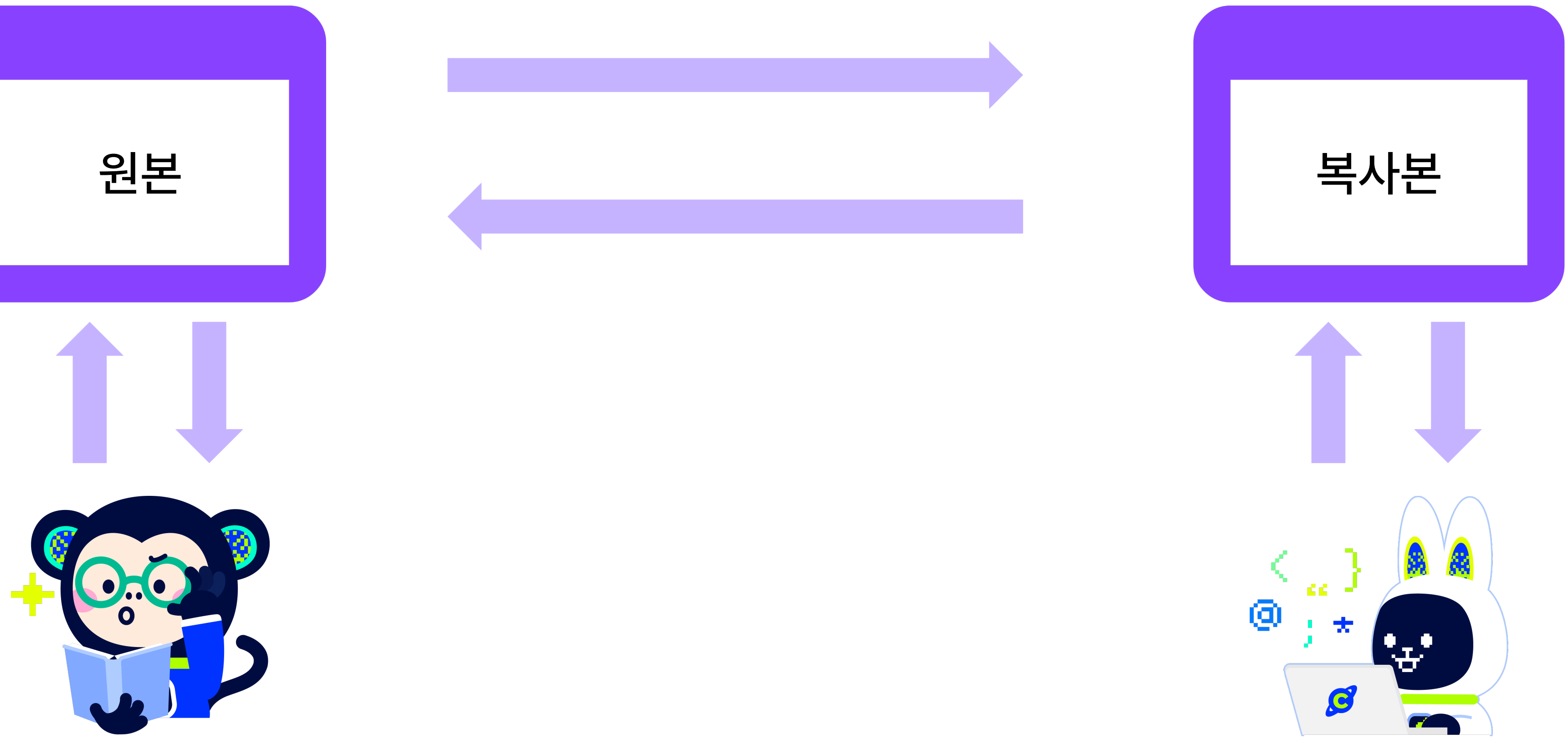
# 연결된 원격 저장소 확인
$ git remote -v
origin https://github.com/team/repo.git

# 원격 저장소로 브랜치 푸시
$ git push -u origin feature/readme
Enumerating objects: 5, done.
Writing objects: 100% (3/3), 412 bytes | 412.00 KiB/s
remote: Create a pull request for 'feature/readme' on GitHub:
remote:   https://github.com/team/repo/pull/new/feature/readme
* [new branch]      feature/readme -> feature/readme
branch 'feature/readme' set up to track 'origin/feature/readme'
```

- 1 git push — 원격 저장소로 전송**
로컬 브랜치의 커밋을 원격 저장소(**origin**)로 업로드합니다.
이제 GitHub에서도 브랜치와 커밋을 확인할 수 있습니다.
- 2 -u 옵션 — 추적 관계 설정**
-u (또는 **--set-upstream**) 는 로컬 브랜치와 **원격 브랜치**를 연결합니다.
한 번 설정해두면 이후엔 **git push** / **git pull** 만으로 동기화됩니다.
- 3 Pull Request 생성 안내**
새 브랜치를 처음 푸시하면 GitHub가 **PR 생성 URL**을 자동으로 알려줍니다.
그 링크를 열거나 GitHub 저장소에서 **"Compare & pull request"** 버튼으로 PR을 만듭니다.

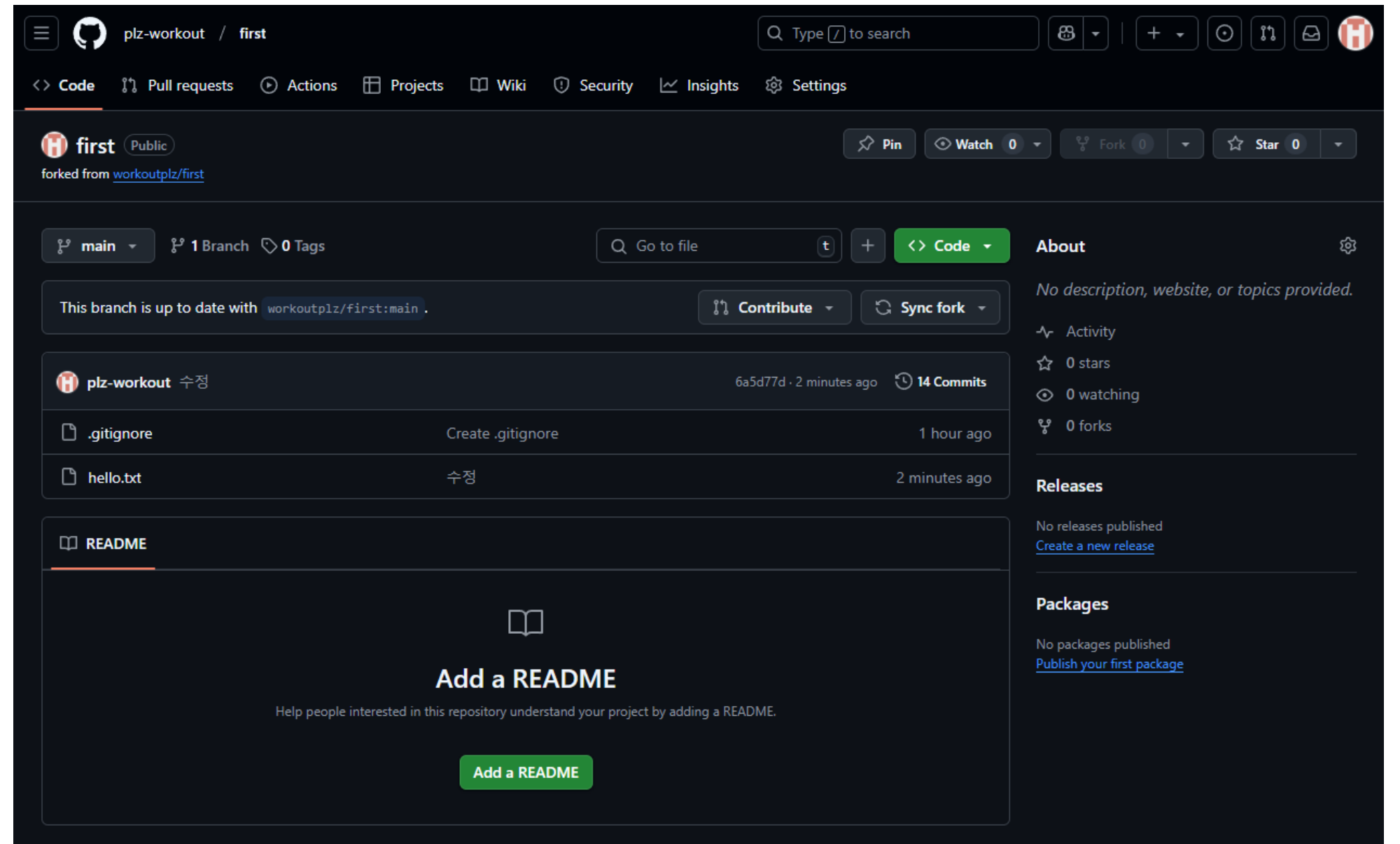
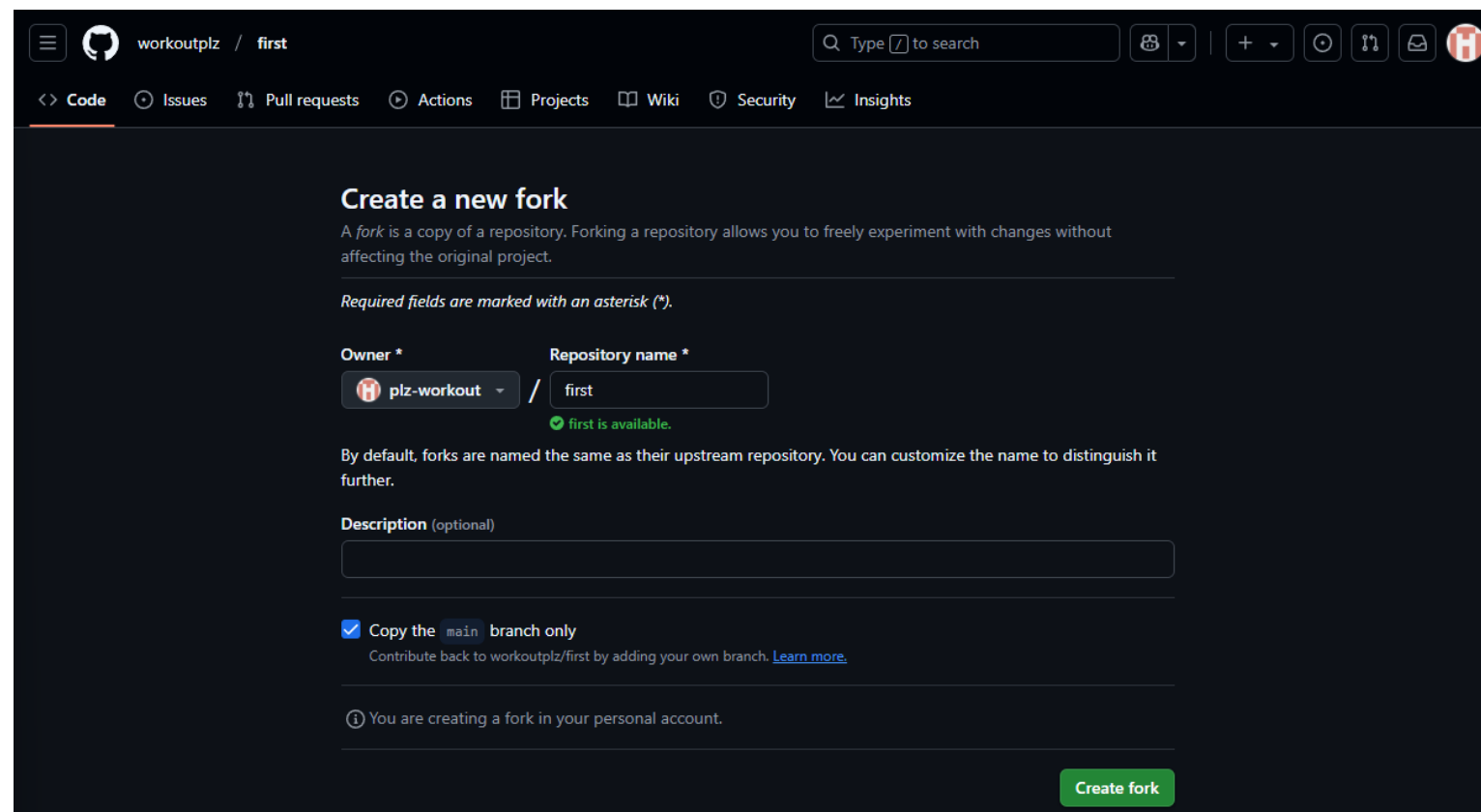
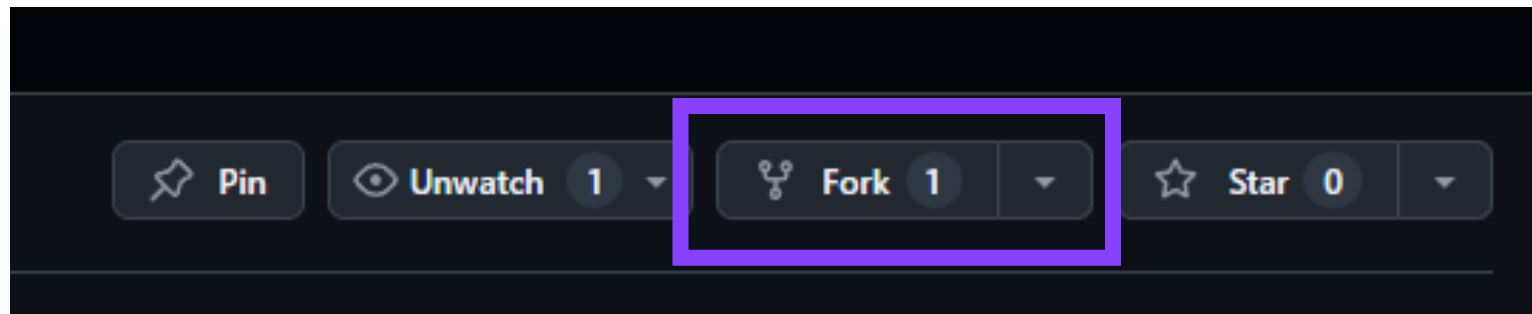


Branch 생성과 Pull Request



내용은 같으나 **따로 존재하는 Remote Repository**를 각자 직접 수정, 통합

Branch 생성과 Pull Request

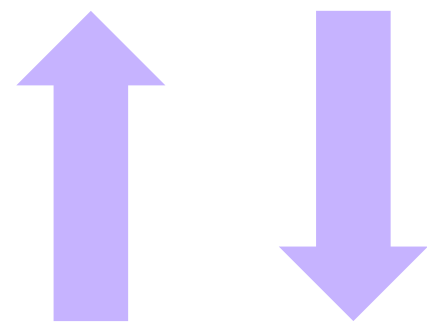
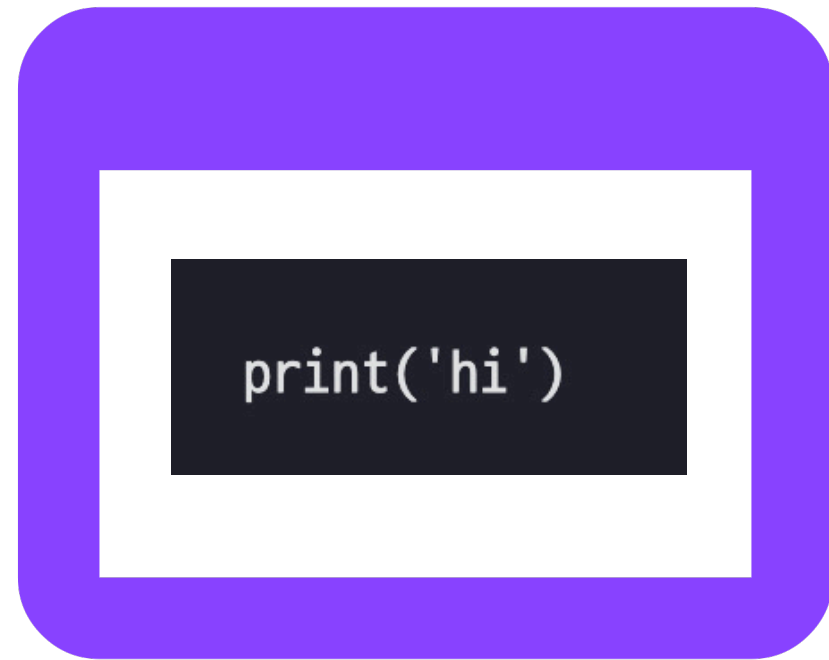


Fork = 내용이 동일한 Repository를 내 계정으로 복사

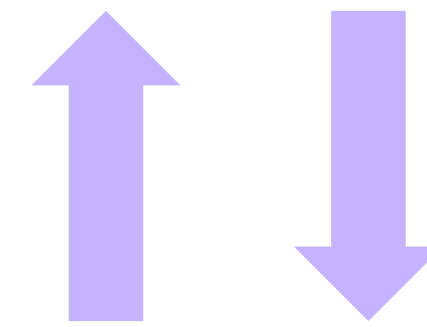
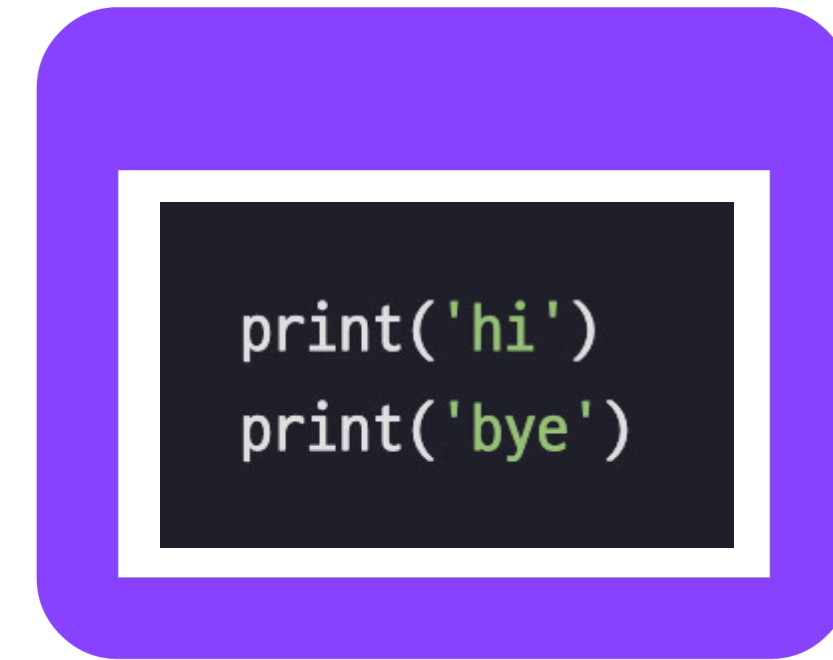


Branch 생성과 Pull Request

원본



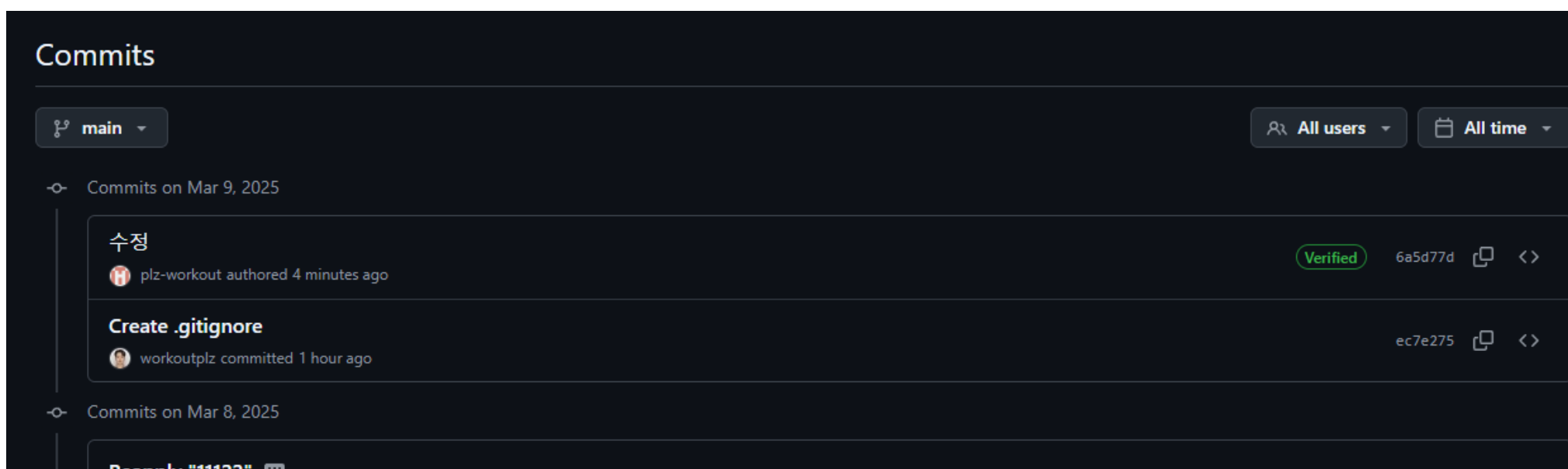
복사본



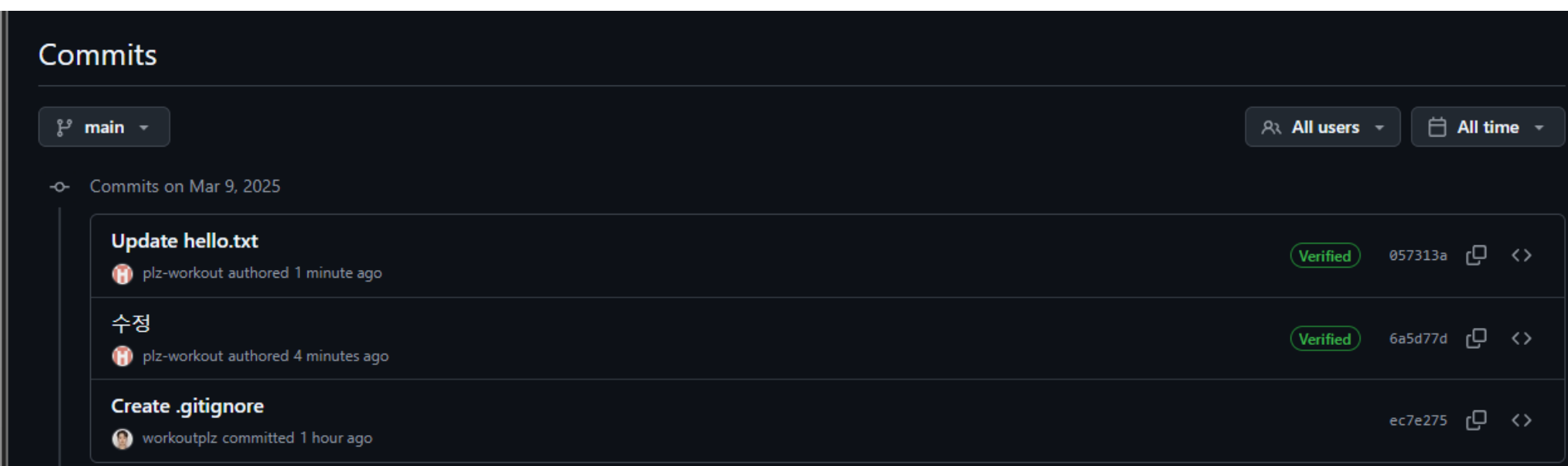
각각의 개발자는 본인의 Repository를 수정하므로 서로의 수정 내용이 반영되지 않음



Branch 생성과 Pull Request



기존 Repository

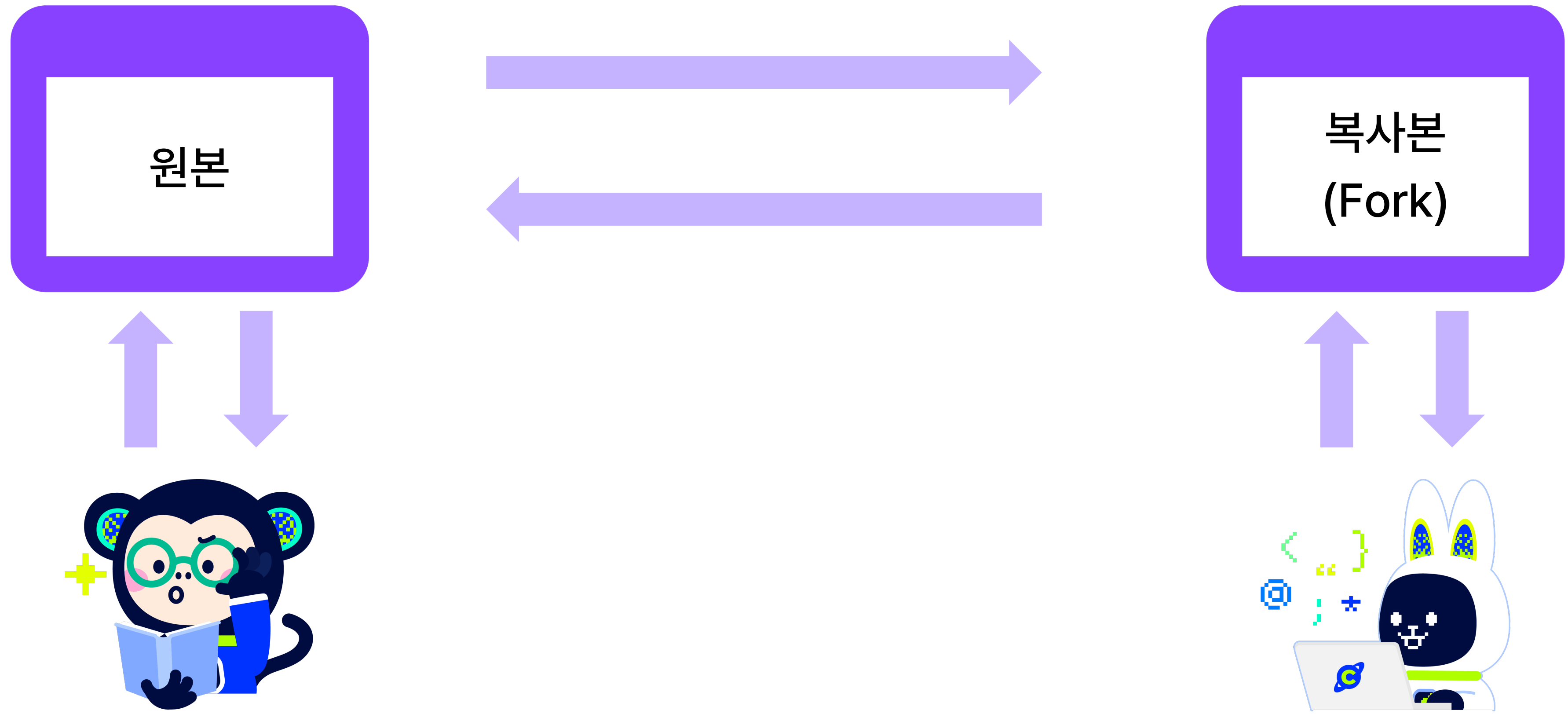


Forked Repository

Fork 한 Repository에서 수정을 해도 원래의 Repository에는 반영이 안되는 모습



Branch 생성과 Pull Request

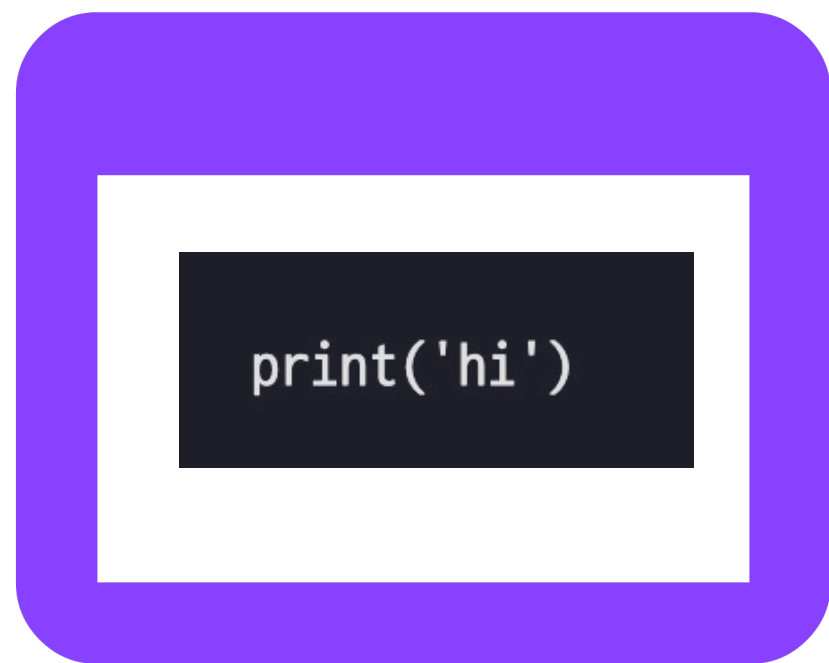


그리고 이 둘을 동기화 시켜 주는 것이 Pull Request와 Merge

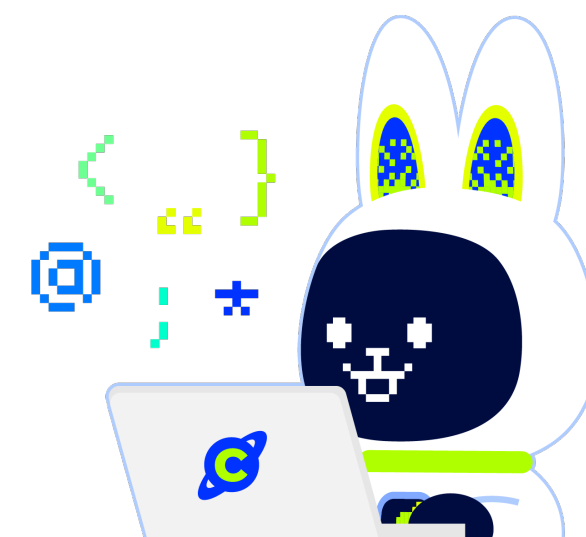
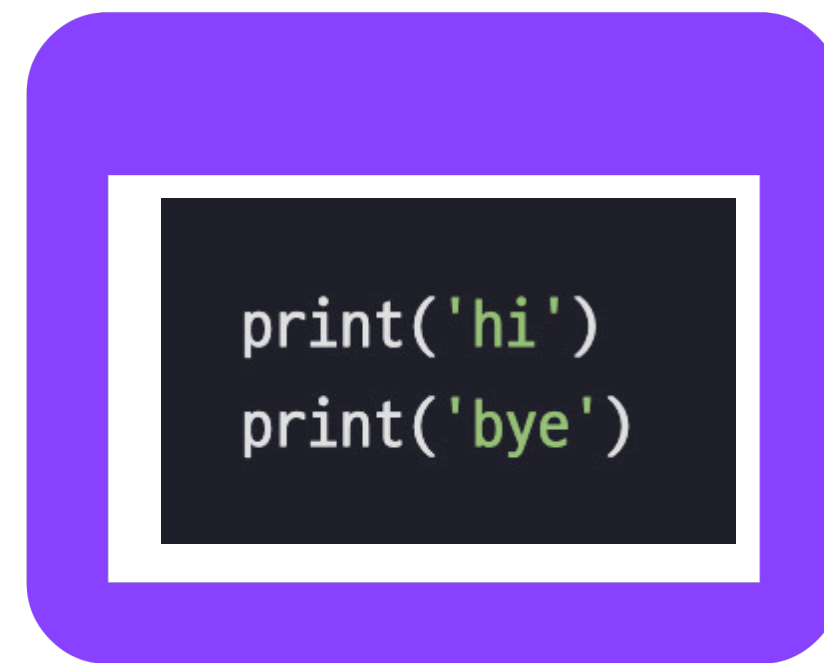


Branch 생성과 Pull Request

원본



복사본



Pull Request

복사본에서 통합 요청을 하는 것 = Pull Request (PR)



Branch 생성과 Pull Request

원본

```
print('hi')  
print('bye')
```



복사본

```
print('hi')  
print('bye')
```



Pull Request

Pull Request를 승인 시 Merge 되어 원본 또한 바뀜



풀 리퀘스트(Pull Request, PR)(1/2)

- 브랜치에서 작업한 코드 변경사항을 팀에 공유하고, 병합 전 검토(리뷰)를 받는 협업 절차
- 병합 전에 코드의 품질을 검증하고, 실수를 방지하며, 팀원과 소통하는 중요한 중간 단계
- 왜 필요한가?
 - 코드 품질 향상 : 리뷰 과정을 통해 버그, 스타일, 로직 오류 등을 사전에 발견
 - 협업 안전성 확보 : 직접 main 브랜치에 병합하지 않음으로써 안정성 보장
 - 기록 추적 : PR에 남는 토론/리뷰 기록으로 변경 이유와 결정 과정을 추적 가능



풀 리퀘스트(Pull Request, PR)(2/2)

단계	설명
1단계 작업 브랜치 생성	feature / 로그인, bugfix / 404-error 등 개별 브랜치에서 개발 진행
2단계 코드 작업 완료	로컬에서 커밋 및 푸시
3단계 Pull Request 생성	GitHub, GitLab 등에서 PR 생성
4단계 코드 리뷰	팀원들이 변경 내용을 확인하고 코멘트 / 피드백 제공
5단계 코드 수정(필요 시)	리뷰 반영하여 다시 커밋 / 푸시
6단계 승인 및 병합	리뷰어 승인 후 main 브랜치에 병합 (Merge / Squash / Rebase 등)



Branch 생성과 Pull Request

[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [Security](#) [Insights](#)

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#) or [learn more about diff comparisons](#).

base repository: workoutplz/first

base: main

← ...

head repository: plz-workout/first

compare: main

✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#) [Create pull request](#)

↻ 1 commit

± 1 file changed

👤 1 contributor

Commits on Mar 9, 2025

Update hello.txt

plz-workout authored now

Verified 057313a <>

Showing 1 changed file with 1 addition and 0 deletions. [Split](#) [Unified](#)

1 hello.txt

...	...	@@ -1,2 +1,3 @@
1	1	1112
2	2	수정 수정
3		+ fork 이후 수정

commit을 기준으로
Pull Request를 생성할 수 있음



Branch 생성과 Pull Request

The screenshot shows a GitHub Pull Request (PR) titled "Update hello.txt #1". The PR is open, showing a diff of changes to the file "hello.txt". The PR was created by "plz-workout" and is targeting the "main" branch of the "workoutplz" repository. The PR description includes a comment in Korean: "hello.txt에 고칠 것이 있어서 조금 수정했습니다." (I made a small modification to hello.txt because I needed to fix it). The PR is currently in the "Open" state. The interface shows the PR details, including the commit history, the diff, and the review status. The PR is currently pending review, and the reviewer "workoutplz" has not yet provided a review. The PR is also marked as "Verified". The PR is currently in the "Open" state, and the reviewer "workoutplz" has not yet provided a review. The PR is also marked as "Verified". The PR is currently in the "Open" state, and the reviewer "workoutplz" has not yet provided a review. The PR is also marked as "Verified".

The screenshot shows the GitHub Pull Request list page for the "workoutplz / first" repository. The page displays a list of pull requests, with the first one being "Update hello.txt" (PR #1) by "plz-workout". The PR is in the "Open" state. The page also shows a "Filters" section with a search bar and various filters like "Labels", "Milestones", "Author", "Label", "Projects", "Milestones", "Reviews", "Assignee", and "Sort". The "New pull request" button is visible in the top right corner. The page also includes a "ProTip!" section with a link to "base:main".

Pull Request가 오면 원작자는 해당 내용을 리뷰 후
Commit 내용을 Merge 할 수 있음



Branch 생성과 Pull Request

Commits

main

All users All time

Commits on Mar 9, 2025

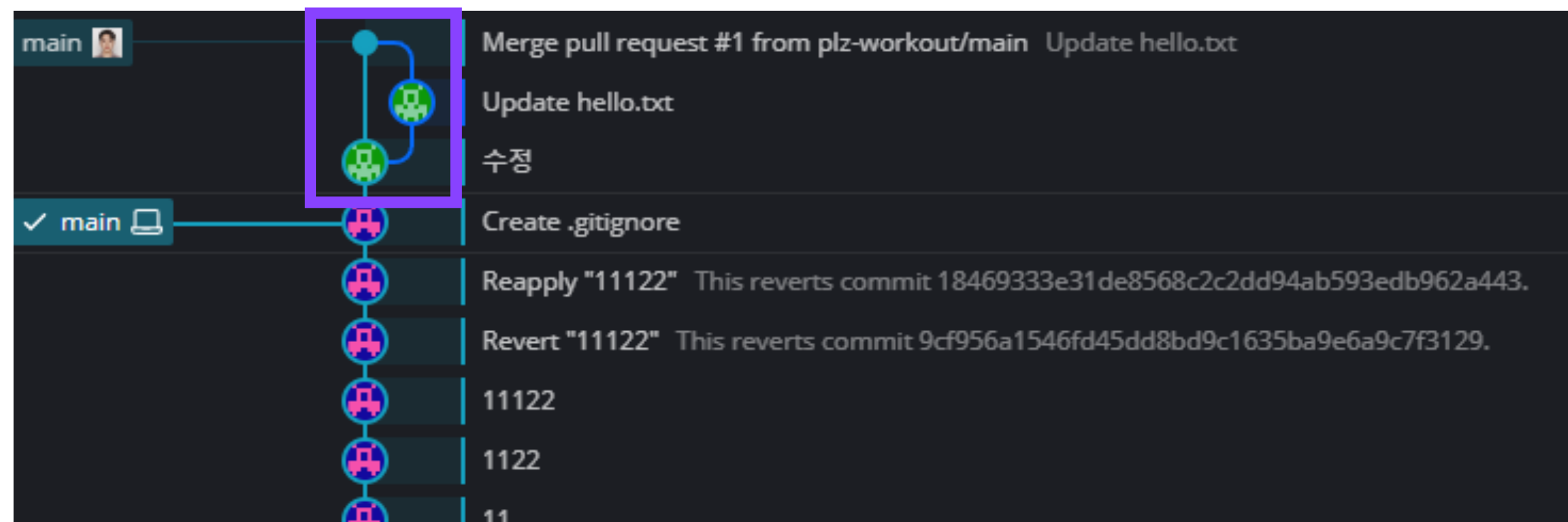
Merge pull request #1 from plz-workout/main	Verified	7dda59a		<>
workoutplz authored now				
Update hello.txt	Verified	057313a		<>
plz-workout authored 4 minutes ago				
수정	Verified	6a5d77d		<>
plz-workout authored 8 minutes ago				
Create .gitignore		ec7e275		<>
workoutplz committed 1 hour ago				
Commits on Mar 8, 2025				
Reapply "11122"		f9e88a3		<>
workoutplz committed 1 hour ago				

Merge가 되면 정상적으로 Commit 내용들이 모두 정상적으로 반영 됨



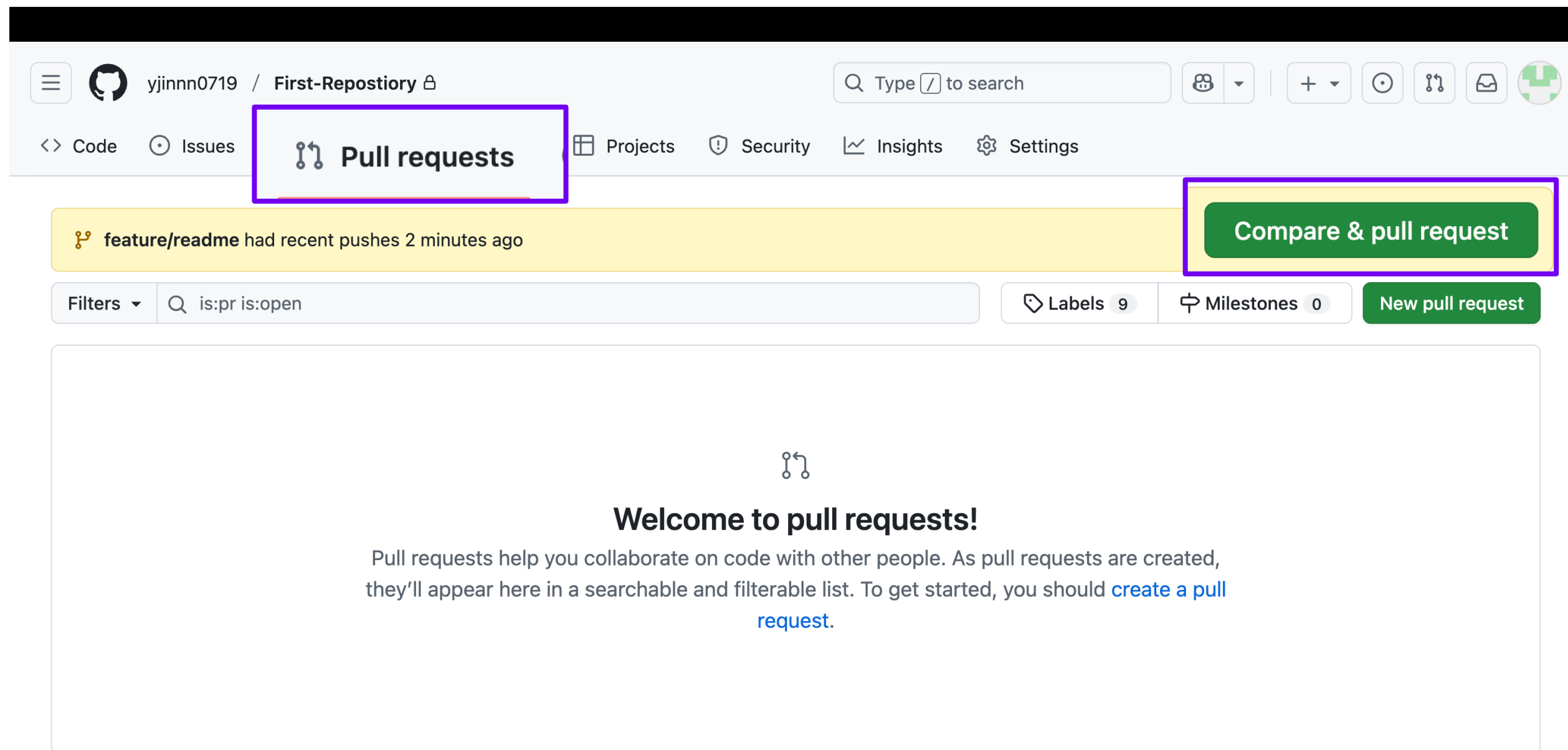
Branch 생성과 Pull Request

Pull Request & Merge 시점



변경 내용을 시각화 하면 위와 같이 나타낼 수 있음
(위 프로그램은 GitKraken)

Pull Request 하기



ProTip! Find everything you created by searching [author:yjinnn0719](#).

1. 상단바에서 Pull requests
클릭

2. 우측 상단
Compare & pull request
클릭

Pull Request 하기

Open a pull request

Create a new pull request by comparing changes across two branches. If you need to, you can also [compare across forks](#). [Learn more about diff comparisons here](#).

 base: main  compare: feature/readme

... will be automatically merged.



Add a title

Update README.md

Add a description

Write Preview H B I          

README에 본 Repository에 대한 개요를 추가하였습니다.

 Markdown is supported

 Paste, drop, or click to add files

Create pull request

 Remember, contributions to this repository should follow our [GitHub Community Guidelines](#).

Reviewers

No reviews

Assignees

No one—[assign yourself](#)

Labels

None yet

Projects

None yet

Milestone

No milestone

Development

Use [Closing keywords](#) in the description to automatically close issues

Helpful resources

[GitHub Community Guidelines](#)

1. 좌측 상단의 브랜치 확인

2. Add a description 입력

3. Create pull request 클릭

base

최종적으로 변경사항이 들어가야 하는 브랜치

compare

최종적으로 변경사항이 들어가야 하는 브랜치

Pull Request 하기

The screenshot shows a GitHub Pull Request interface. At the top, the repository is 'yjinnn0719 / First-Repository'. The pull request title is 'Update README.md #1'. Below the title, it says 'yjinnn0719 wants to merge 1 commit into main from feature/readme'. The interface includes tabs for Conversation (0), Commits (1), Checks (0), and Files changed (1). A comment from yjinnn0719 is visible, stating 'README에 본 Repository에 대한 개요를 추가하였습니다.' Below the comment, there is a green box indicating 'No conflicts with base branch' and a 'Merge pull request' button. The right sidebar shows options for adding reviewers, labels, projects, milestones, and development. At the bottom, there is a section for 'Add a comment' with a rich text editor.

1. 충돌이 없다면 Merge pull request 클릭

필요 시, Pull Request를 검토할 리뷰어를 지정할 수 있다.

Pull Request 하기

yjinnn0719 / First-Repository

Q Type to search

+

<> Code

Issues

Pull requests

Actions

Projects

Security

Insights

Settings

Update README.md #1

Edit <> Code

Merged

yjinnn0719 merged 1 commit into main from feature/readme now

Conversation 0

Commits 1

Checks 0

Files changed 1

+3 -1

Changes from all commits

File filter

Conversations

Jump to

Review in codespace

Review changes

Update README.md

main (#1)

yjinnn0719 authored 16 minutes ago Verified commit 15edafef2024158d7085273feb9ebf303028f13c

4 README.md

<> File Comment

@@ -1 +1,3 @@

1 - # First-Repository

1 + # First Repository

2 + **Git & GitHub 학습용 저장소**입니다.

3 + 브랜치, Pull Request, merge 등 협업 기능을 실습합니다.

1. 상단바 → Commits 클릭

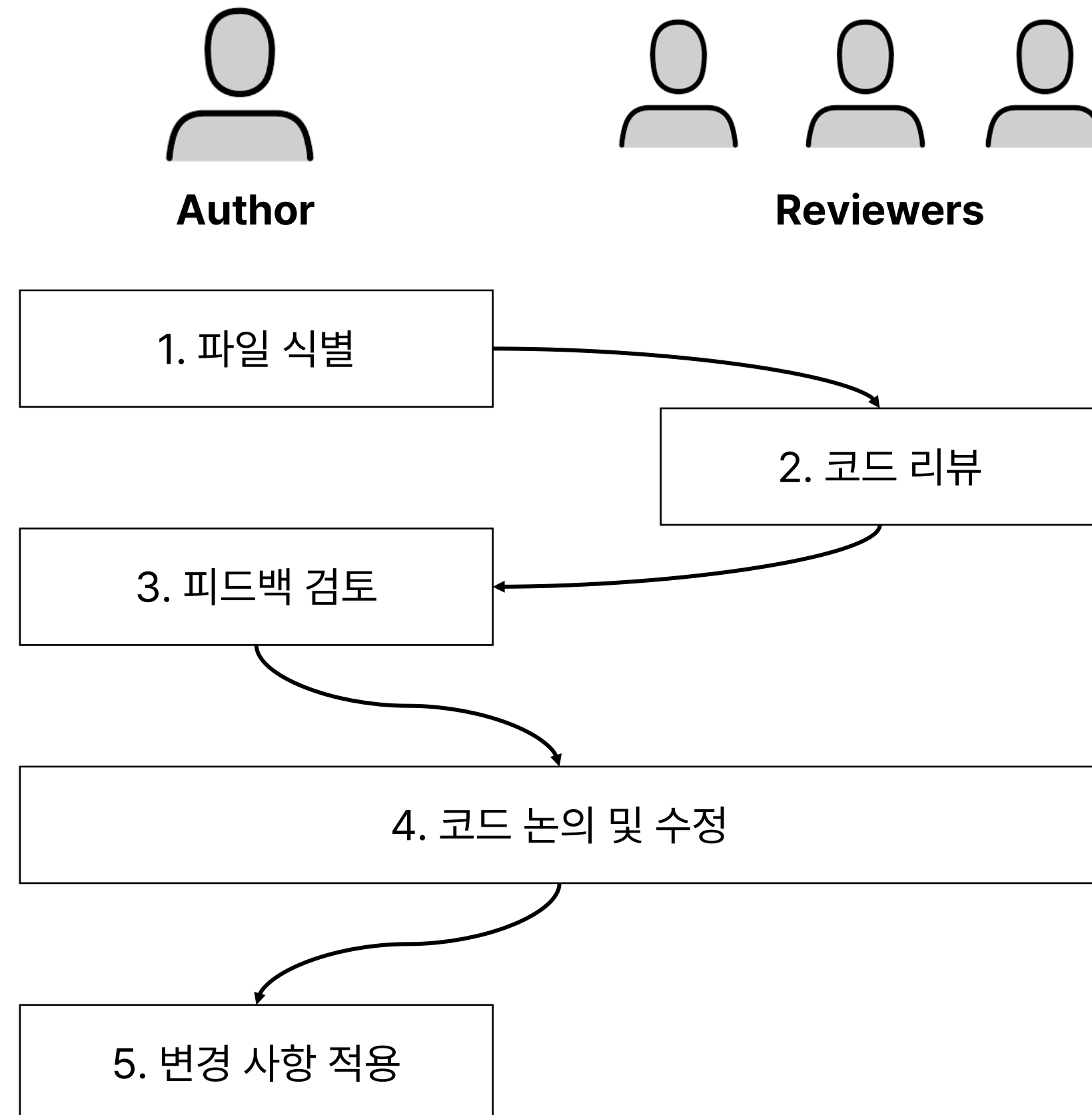
2. 내용 확인



MR(PR) 작성 매너(1/2)

코드 리뷰 기본 원칙

- 모든 코드는 리뷰 후 머지
- 리뷰 책임은 팀 전체가 가짐
- 최소 1명의 리뷰어 승인 필수
- PR 요청 시 하루 안에 답변
- 양이 많으면 오프라인 리뷰 요청
- 사실과 데이터로 의견 제시



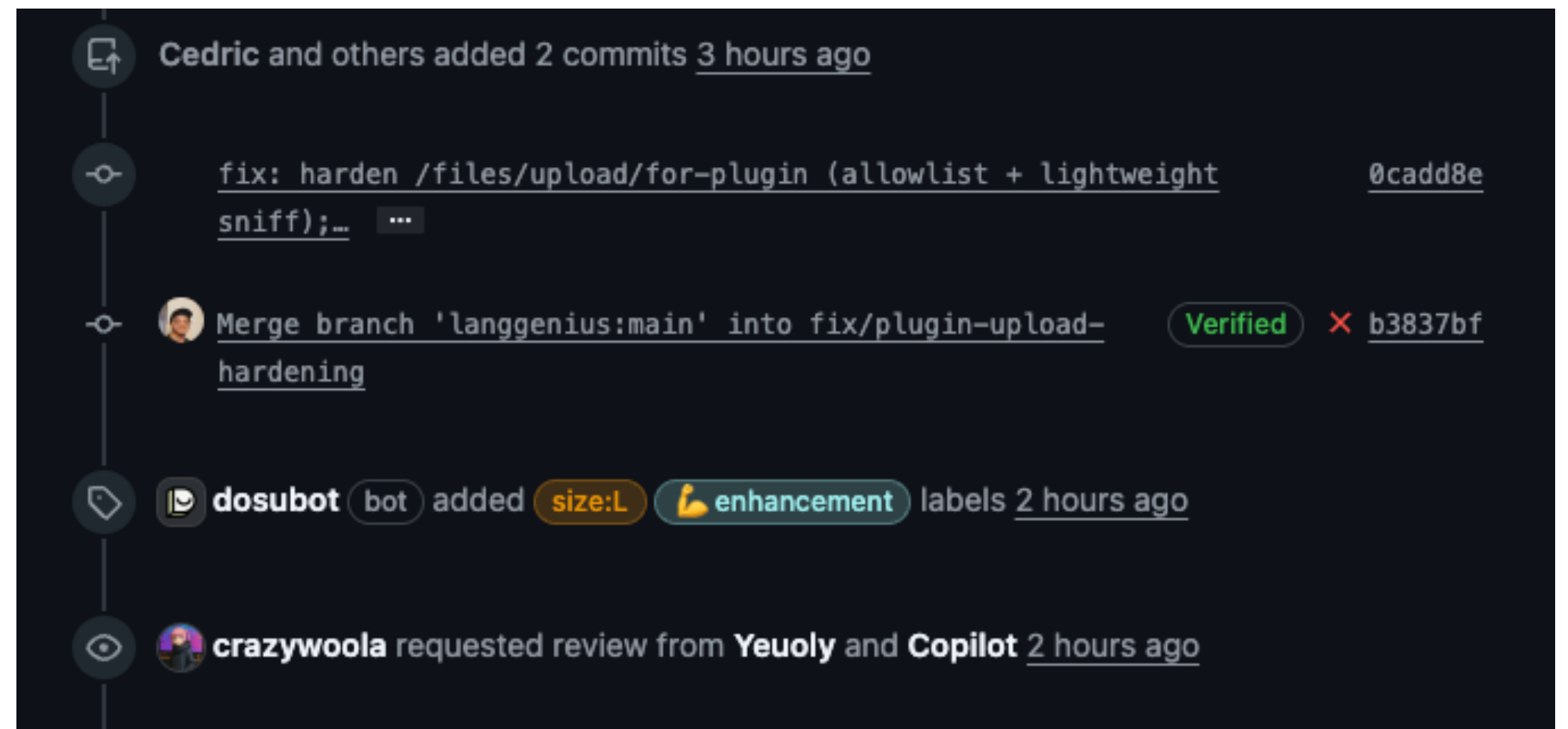


Branch 생성과 Pull Request

MR(PR) 작성 매너(2/2)

PR 요청 시 매너

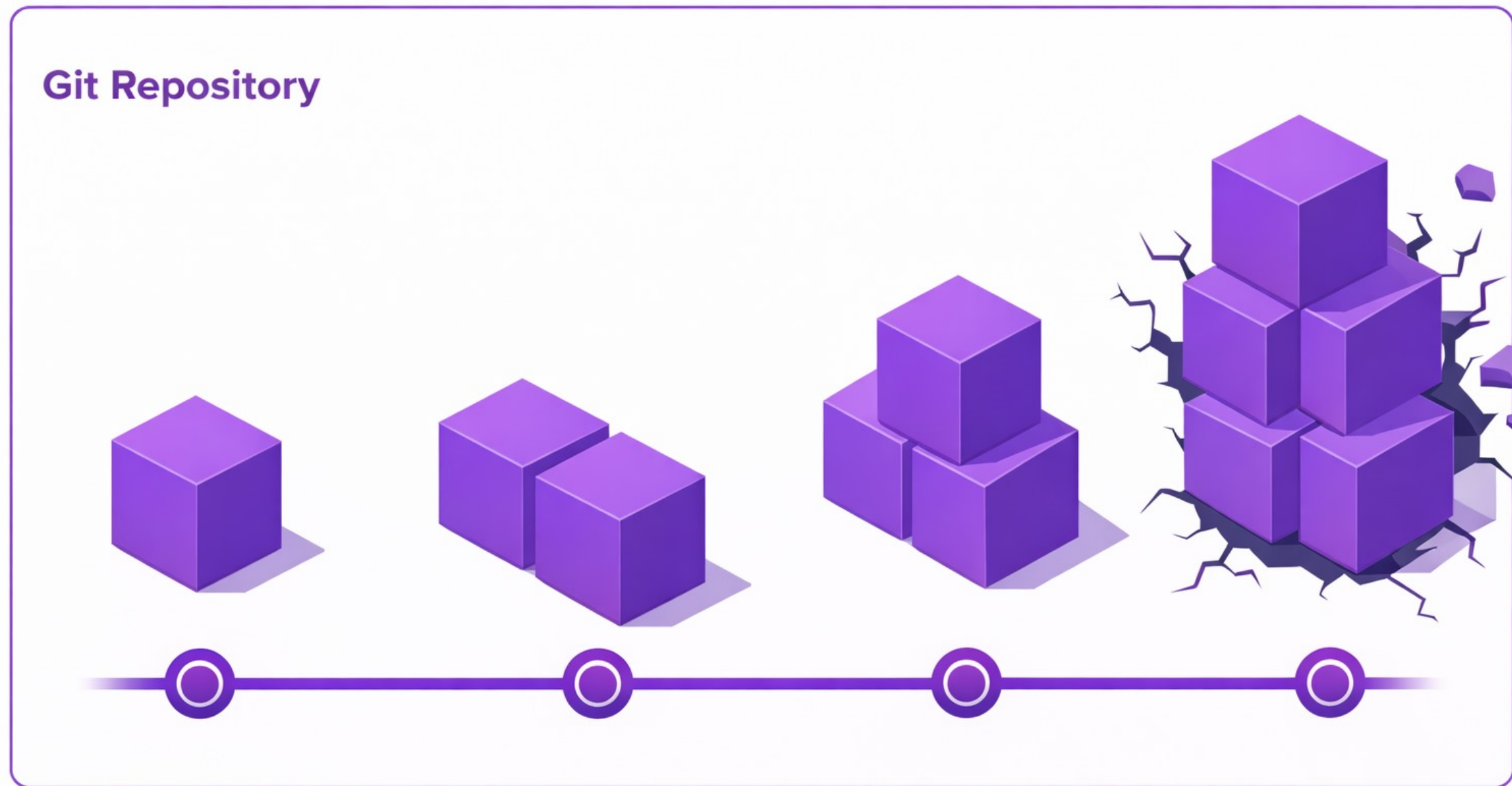
- 변경 내용 충분히 설명
- PR 제목 : (키워드) 작업 내용 - PR 순서
- 본문 : 작업 동기, 변경 내용, 주안점, 연관 PR/티켓, 체크리스트 포함
- 스크린샷/GIF 첨부 (UI 변경 시)
- 작은 단위로 PR 생성
- 주석 / 커밋 / 본문 모두 충실
- 중복 없이 관련 코드 포함



Git LFS 개념 및 활용 방법



기존 Git의 구조적 한계 : Snapshot



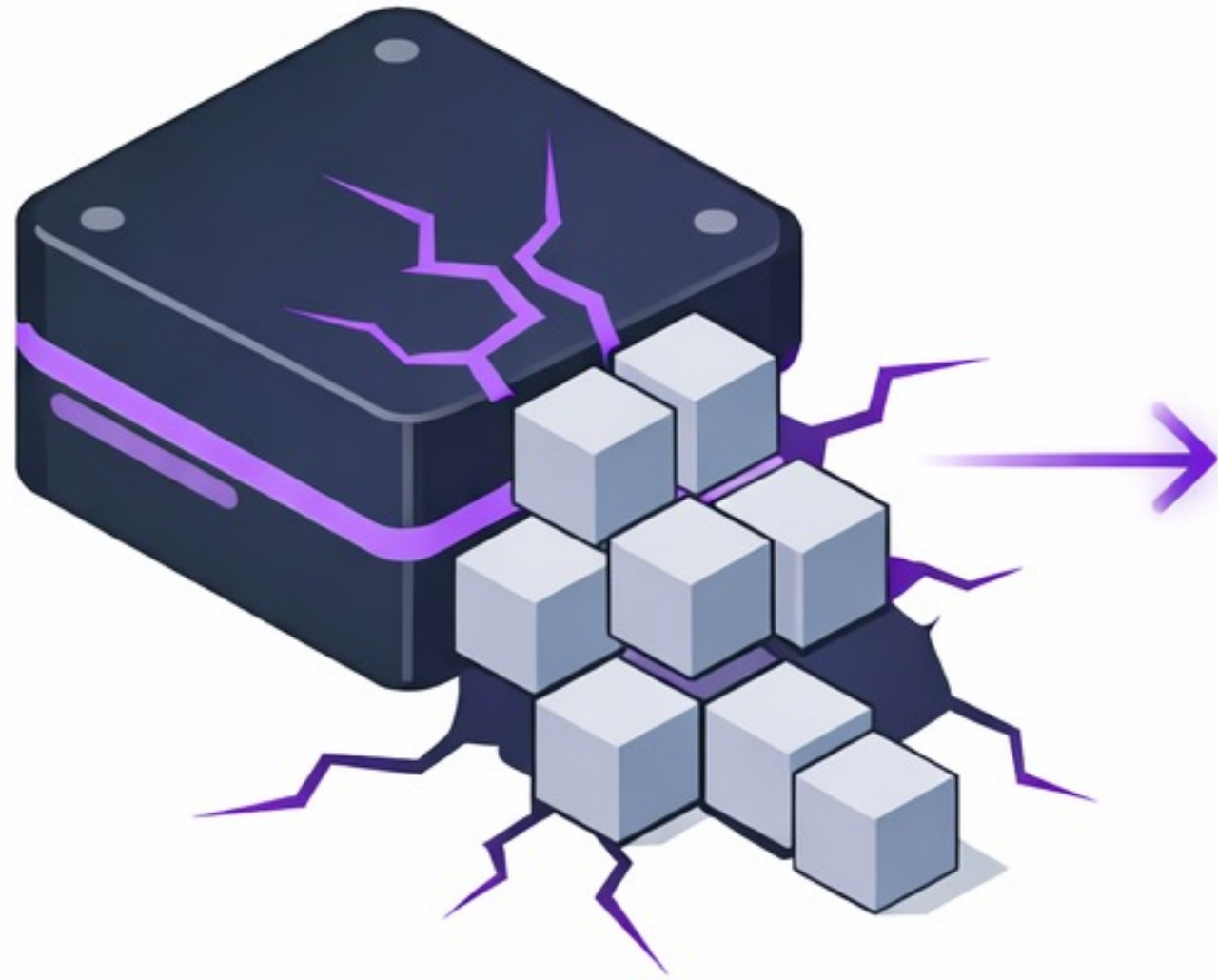
Snapshot 방식

Git은 커밋마다 전체 트리를 **스냅샷**으로 기록하지만, 변경되지 않은 파일은 **기존 객체를 그대로 참조**하므로 텍스트 파일은 효율적으로 관리됨

단, **이미지·영상·바이너리처럼 diff 효율이 떨어지는 파일**은 작은 수정에도 거의 전체가 새 객체로 저장 → **매 커밋마다 사실상 복제**되어 저장소 용량이 폭증



기존 Git의 구조적 한계 : Snapshot



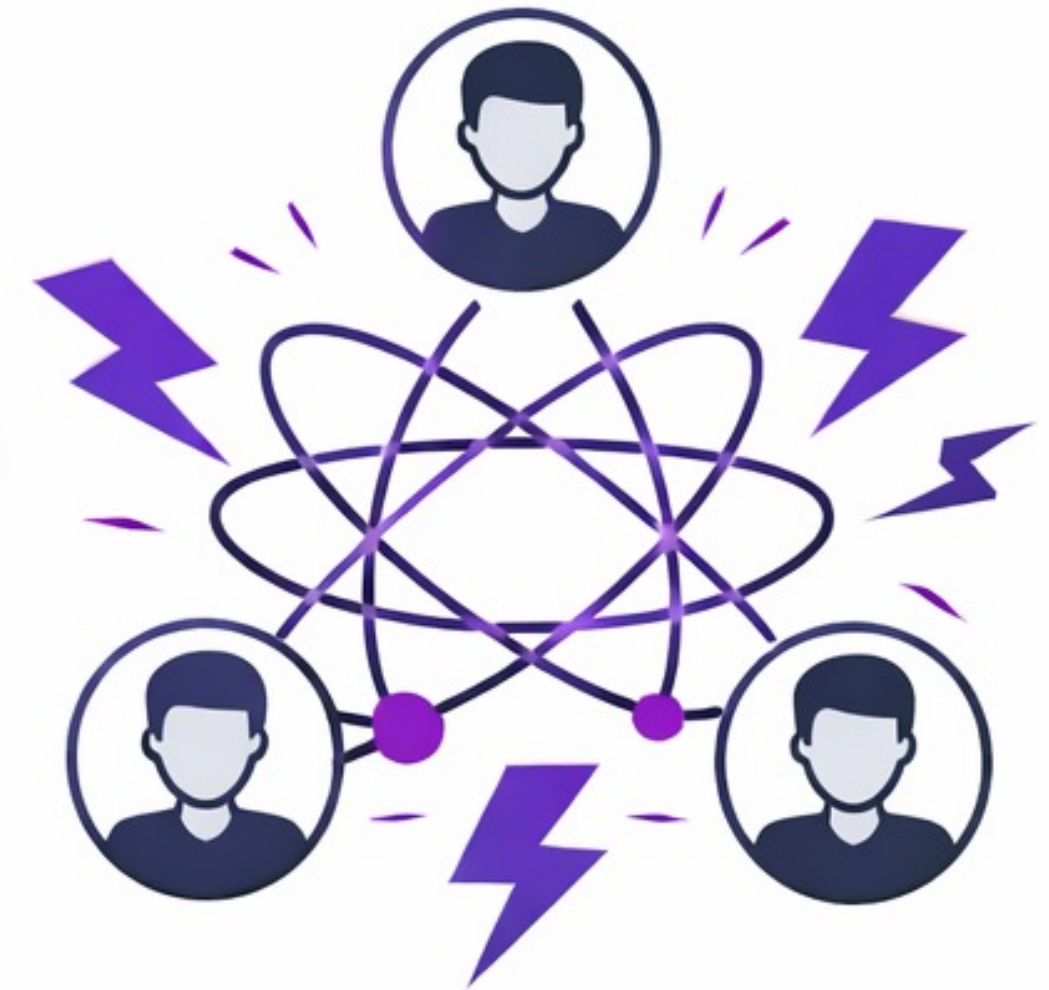
저장소 용량 급증

단일 대용량 파일 하나가 전체 레포지토리 크기를 압도합니다.



작업 속도 저하

파일 크기에 비례하여 Commit, Push, Clone에 소요되는 시간이 급격히 늘어납니다.



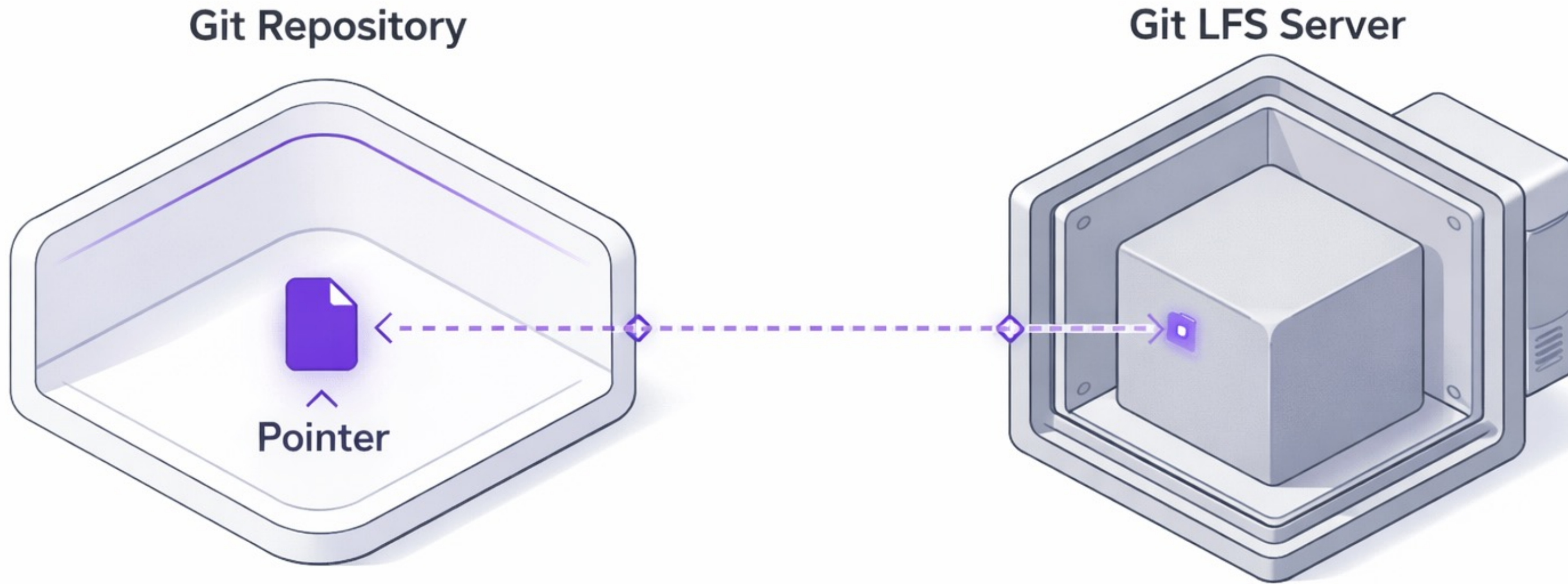
불필요한

히스토리 강제 공유

협업 시 모든 팀원이 원치 않는 대용량 파일의 변경 이력까지 강제로 다운로드해야 합니다.



Git LFS 개념 및 활용 방법



Git LFS(Large File Storage)

Git이 효율적으로 처리하지 못하는 대용량 파일을 다루기 위한 공식 확장 도구

외부 격리 저장

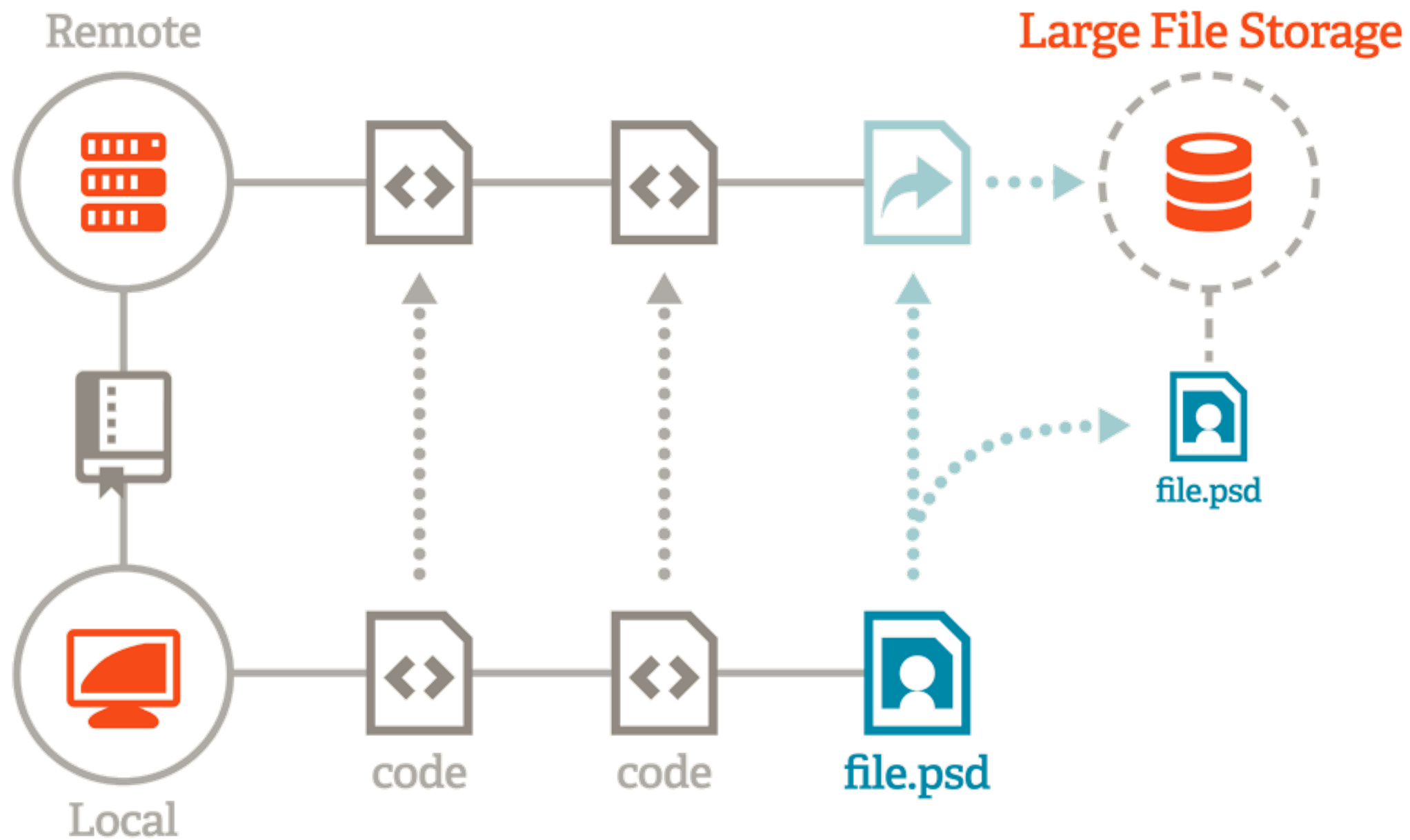
실제 대용량 파일은 Git 저장소 외부에 별도로 보관

포인터(Pointer) 파일

Git 내부 히스토리에는 원본 파일의 위치와 해시값만 담긴 수십 바이트 수준의 가벼운 포인터 파일만 저장됨



Git LFS 동작 구조



로컬(Local) : 개발자는 코드와 함께 대용량 파일의 참조 정보만 Git에 커밋함

리모트(Remote) : 원격 Git 서버는 참조만, 실제 대용량 파일은 LFS 서버에 저장



Git LFS 설치 (OS별)

운영체제	설치 명령어	초기화 / 비고
Mac OS	<code>brew install git-lfs</code>	<code>git lfs install</code> (사용자 환경 초기화 - 한 번만)
Windows	<code>choco install git-lfs</code> 또는 <code>winget install git-lfs</code>	<code>git lfs install</code>
Linux(Debian/Ubuntu)	<code>sudo apt-get update</code> <code>sudo apt-get install git-lfs</code>	<code>git lfs install</code>



Git LFS 특징

- 예를 들면, 게임 개발 프로젝트에서 고해상도 텍스처나 3D 모델과 같은 큰 파일을 관리해야 할 때,
- 프로젝트 저장소 크기를 줄이고, 파일을 다운로드하고 업데이트하는 데 걸리는 시간을 단축할 수 있음

대용량 파일 버전 관리	파일 크기에 제한 없이 대용량 파일을 버전 관리할 수 있도록 해주며 GB 단위의 파일도 문제없이 관리할 수 있음
저장소 크기 감소	파일의 포인터만 저장되므로 저장소 크기를 줄일 수 있으며 복제하거나 전송하는 데 걸리는 시간이 단축됨
Git 워크플로우 유지	Git의 기본 워크플로우를 유지하면서 대용량 파일을 관리할 수 있도록 해주며 파일 관리가 투명하게 이루어짐
다양한 호스팅 지원	GitHub, GitLab, Bitbucket 등 다양한 Git 호스팅 서비스에서 지원



Git LFS 대안 비교

- Git LFS : 대용량 파일을 포인터로 관리하여 저장소에 포함되지 않고 사용이 편리하며 협업에 적합함
- Git Submodule : 대용량 파일을 지원하지 않고 파일은 저장소에 포함되며 사용성과 협업 적합도가 낮음
- Git-annex : 대용량 파일을 저장소 외부에 보관하고 사용법이 복잡하며 협업에 적합하지 않음

기능	Git LFS	Git Submodule	Git-annex
대용량 파일 지원	Y	X	Y
히스토리 단순성	좋은(포인터)	보통	보통
사용 난이도	쉬움	중간	어려움
협업 적합도	높음	중간	낮음 (복잡함)

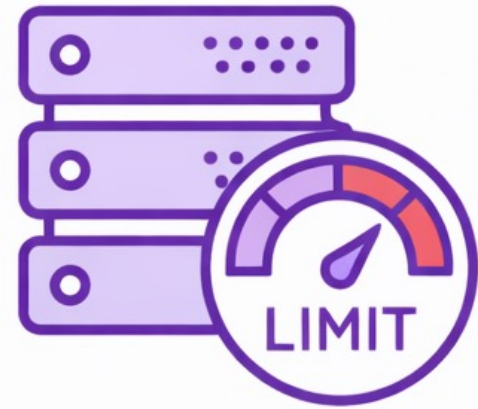


Git LFS 실무 활용 정리

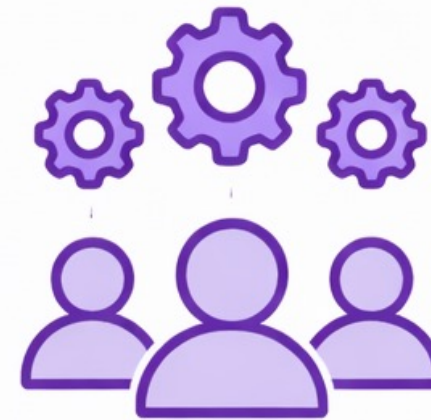
대용량 파일 버전 관리	.mp4, .psd, .zip, .csv 등 Git이 비효율적인 파일을 LFS로 추적
포인터 기반 저장	Git에는 포인터 파일만 저장되고, 파일은 LFS 서버에 저장됨
협업 시 자동 동기화	git pull 시 포인터 기반으로 자동으로 대용량 파일도 함께 내려받음
빠른 저장소 클론/푸시	파일을 나중에 개별 다운로드함으로써 속도 개선
CI/CD 파이프라인 연동	LFS 파일을 자동으로 받아와 테스트/배포에 활용 가능(git lfs pull)
LFS 파일만 개별 다운로드 가능	git lfs fetch, git lfs checkout으로 특정 파일만 받아올 수 있음
포인터 추적 확인	git lfs ls-files, .gitattributes로 추적 상태 점검 가능
저장소 용량 절약 및 관리	불필요한 캐시 정리(git lfs prune), 사용량 확인 가능
주의사항	공동 작업자 모두 git lfs install 필요, LFS 미지원 서버 주의



Git LFS 사용 시 고려사항



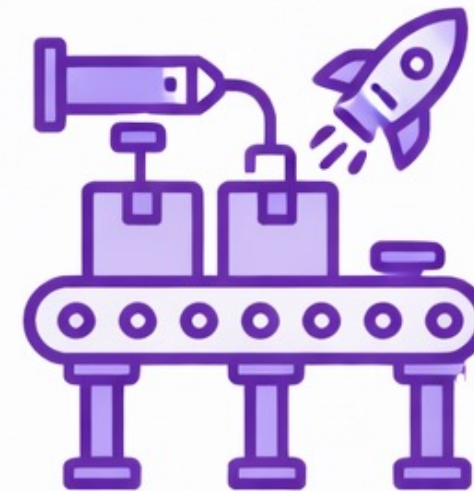
스토리지 및 트래픽 제한



전사적 환경 구성



호스팅 서버 지원 여부



CI/CD 파이프라인 연동

고급 Git 기능



고급 Git 기능



git stash

작업 중인 변경사항을
임시로 저장하고 복원



git cherry-pick

특정 커밋만 골라서
다른 브랜치에 적용



git rebase

커밋 히스토리를
깔끔하게 정리



git reflog

모든 이력 기록 보기
(삭제된 커밋도 복구 가능)



git bisect

이진 탐색으로
버그 원인 커밋 추적

Git Stash : 임시 저장 기능

- 작업 중이던 변경 사항을 임시로 저장하고 브랜치를 이동할 수 있음
- 브랜치 전환 시 미완성 작업 보관

명령어	설명
git stash	변경 사항 임시 저장
git stash list	저장된 stash 목록 보기
git stash apply	마지막 stash 적용 (삭제 안됨)
git stash pop	적용 + 목록에서 제거

```

~/test-project on develop +1
└─> git stash
Saved working directory and index state WIP on develop: cbfc661 chore: add LFS attributes

~/test-project on develop *1
└─> git stash list
stash@{0}: WIP on develop: cbfc661 chore: add LFS attributes

~/test-project on develop *1
└─> git stash apply
현재 브랜치 develop
커밋할 변경 사항:
  (use "git restore --staged <file>..." to unstage)
      새 파일:      test.py

└─> git stash pop
현재 브랜치 develop
커밋할 변경 사항:
  (use "git restore --staged <file>..." to unstage)
      새 파일:      test.py

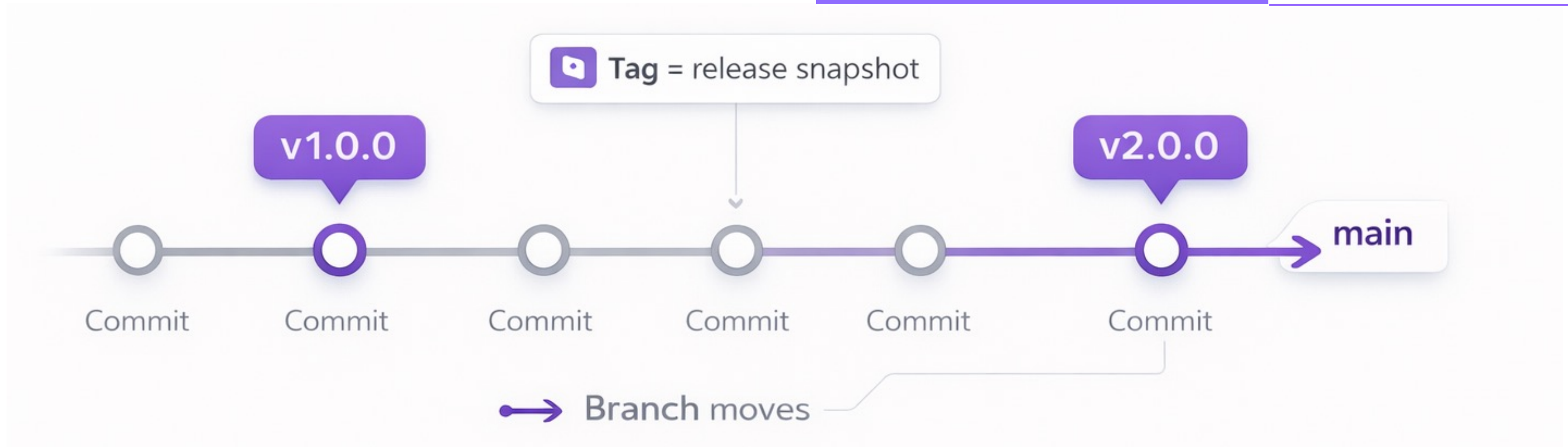
Dropped refs/stash@{0} (d75d6fc8702e50d1171b7dcceb2a549157c0d97e)

```

Git Tag : 버전 식별자 부여

- 커밋에 버전 태그를 지정해 릴리즈 포인트를 명확히 함
- 주로 main 브랜치의 릴리즈 커밋에 부여
- CI/CD 배포 시점 명확히 구분 및 버전 관리

명령어	설명
<code>git tag v1.0.0</code>	로컬에 태그 추가
<code>git push origin v1.0.0</code>	원격에 푸시





Git Cherry-pick : 선택적 커밋 적용

- 특정 커밋만 골라 다른 브랜치에 반영할 수 있음
- Hotfix, 긴급 수정 사항을 다른 브랜치에 빠르게 반영할 때 유용

명령어	설명
git cherry-pick	긴급 패치 커밋을 릴리즈 브랜치에 적용

```

~/test-project on develop +1
> git cherry-pick
사용법 : git cherry-pick [--edit] [-n] [-m <parent-number>] [-s] [-x] [--ff]
        [-S[<keyid>]] <commit>...

또는 : git cherry-pick (--continue | --skip | --abort | --quit)

--quit                되돌리기 또는 커밋 빼오기 연속을 끝냅니다
--continue            되돌리기 또는 커밋 빼오기 연속을 계속합니다
--abort               되돌리기 또는 커밋 빼오기 연속을 취소합니다
--skip                skip current commit and continue
--[no-]cleanup <모드> 메시지에서 공백과 #주석을 지웁니다
-n, --no-commit       자동으로 커밋하지 않습니다
--commit             opposite of --no-commit
-e, --[no-]edit       커밋 메시지를 편집합니다
-s, --[no-]signoff    add a Signed-off-by trailer
-m, --[no-]mainline <이전 -커밋 -번호>
                     이전 커밋 중 메인라인 선택
--[no-]rerere-autoupdate
                     가능하면 인덱스를 재사용한 충돌 해결로 업데이트합니다
--[no-]strategy <전략>
                     병합 전략
-X, --[no-]strategy-option <옵션>
                     병합 전략 옵션
-S, --[no-]gpg-sign[=<키 -ID>]
                     GPG 서명 커밋
-x                   커밋 이름을 뒤에 붙입니다
--[no-]ff             정방향 진행을 허용합니다
--[no-]allow-empty    최초 빈 커밋을 유지합니다
--[no-]allow-empty-message
                     빈 메시지로 커밋을 허용합니다
--[no-]keep-redundant-commits
                     deprecated: use --empty=keep instead
--empty (stop|drop|keep)
                     how to handle commits that become empty

```


Git Reflog : 과거 HEAD 추적

- 브랜치, HEAD, 리셋 등 Git 내부 이동 기록 추적 가능
- 실수로 브랜치 삭제, reset 등 했을 때 복구에 유용

명령어	설명
git reflog	실수로 지운 커밋 / 브랜치 복구

```
~/test-project on develop +1 x HUP ^ system
> git reflog
cbfc661 (HEAD -> develop, origin/main, origin/HEAD, main) HEAD@{0}: reset: moving to HEAD
cbfc661 (HEAD -> develop, origin/main, origin/HEAD, main) HEAD@{1}: checkout: moving from main to develop
cbfc661 (HEAD -> develop, origin/main, origin/HEAD, main) HEAD@{2}: commit: chore: add LFS attributes
cf53ef2 HEAD@{3}: commit: Add: big files via LFS
d9c5bf3 HEAD@{4}: commit: Add: initial commit
416a588 HEAD@{5}: clone: from http://127.0.0.1:58443/root/test-project.git
```




Git Bisect : 버그 발생 지점 추적

- 이진 탐색 방식으로 버그가 발생한 커밋을 추적
- 실수로 브랜치 삭제, reset 등 했을 때 복구에 유용

명령어	설명
git bisect start	대규모 프로젝트에서 특정 커밋 원인 추적

```
~/test-project on @cf53ef2b bisect +1
> git bisect start
A      test.py
이전 HEAD 위치는 cf53ef2 Add: big files via LFS
'develop' 브랜치로 전환합니다
status: waiting for both good and bad commits

~/test-project on @cf53ef2b bisect +1
> git bisect good
cbfc6619af7893f0d667201648cc197b5c719636 is the first bad commit
commit cbfc6619af7893f0d667201648cc197b5c719636 (origin/main, origin/HEAD, main, develop)
Author: cobyoo <shyeon.yoo@gmail.com>
Date:   Fri Aug 8 18:58:42 2025 +0900

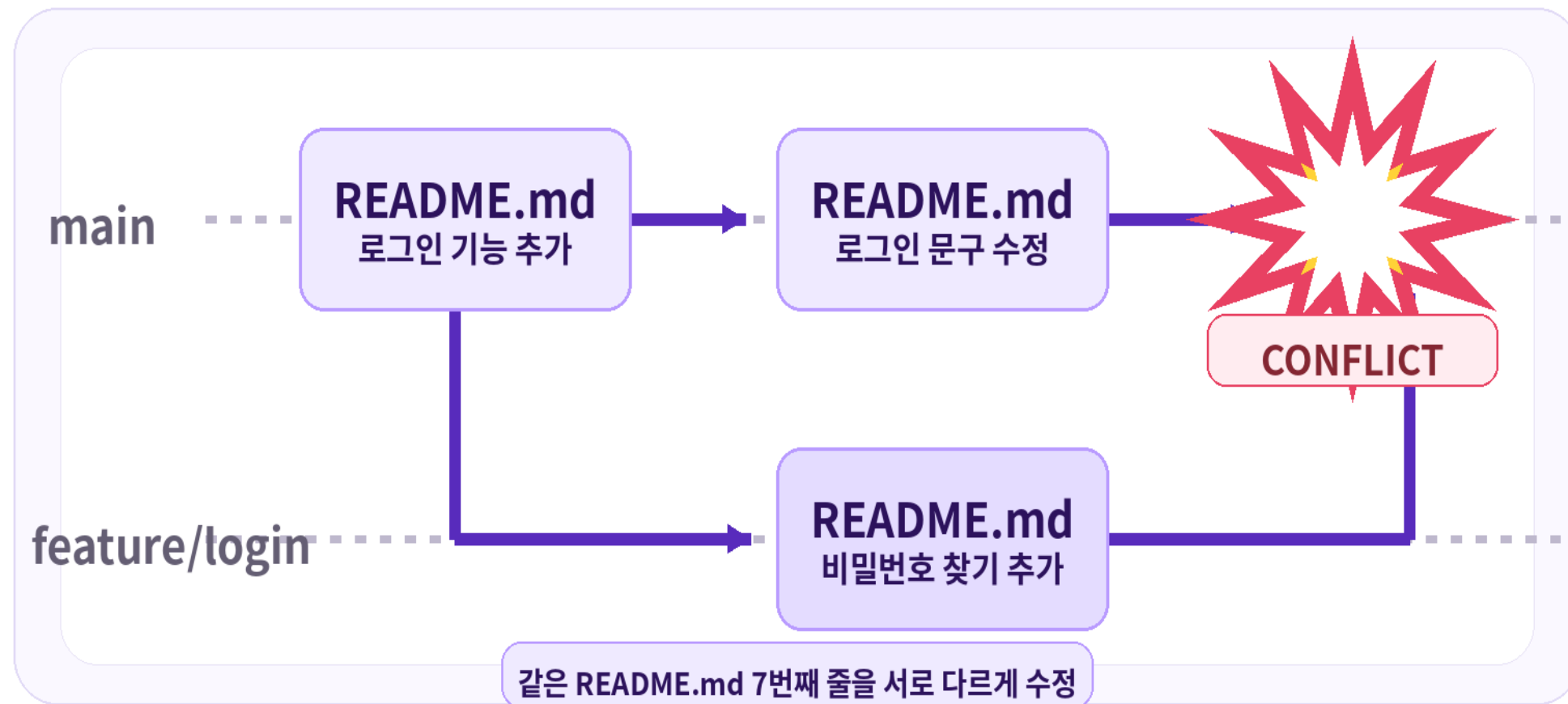
    chore: add LFS attributes

.gitattributes | 2 ++
1 file changed, 2 insertions(+)
create mode 100644 .gitattributes
```

충돌 해결하기

Merge Conflict : 충돌 발생

같은 파일의 같은 라인을 두 브랜치가 서로 다르게 수정하면, Git이 자동으로 합칠 수 없어 **사람에게 결정을 맡깁니다.**



1 같은 줄 동시 수정

두 사람이 같은 파일
같은 줄을 동시에 변경

2 같은 함수 리팩토링

같은 함수를 두 브랜치가
각자 다르게 정리

3 삭제 vs 수정

한쪽에서 삭제한 파일을
다른 쪽에서 수정

4 머지 작업 중

머지 작업 중
자동 병합 실패 ★ **가장 흔함**

VSCode에서의 충돌 마커

```
13
14  /**
15   * Prints the welcome message
16   */
17  Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
18  <<<<<<< HEAD (Current Change)
19  function printMessage(showUsage, message) {
20      console.log(message);
21  }
22  =====
23  function printMessage(showUsage, showVersion) {
24      console.log("Welcome To Line Counter");
25      if (showVersion) {
26          console.log("Version: 1.0.0");
27      }
28  }
29  >>>>>>> theirs (Incoming Change)
30  if (showUsage) {
31      console.log("Usage: node base.js <file1> <file2> ... ");
32  }
33  /**
```

Git이 자동으로 파일에 삽입해주는 마커

<<<<<<< HEAD

충돌 시작 : 여기부터 아래 구분선
까지가 현재 브랜치(HEAD)의 코드

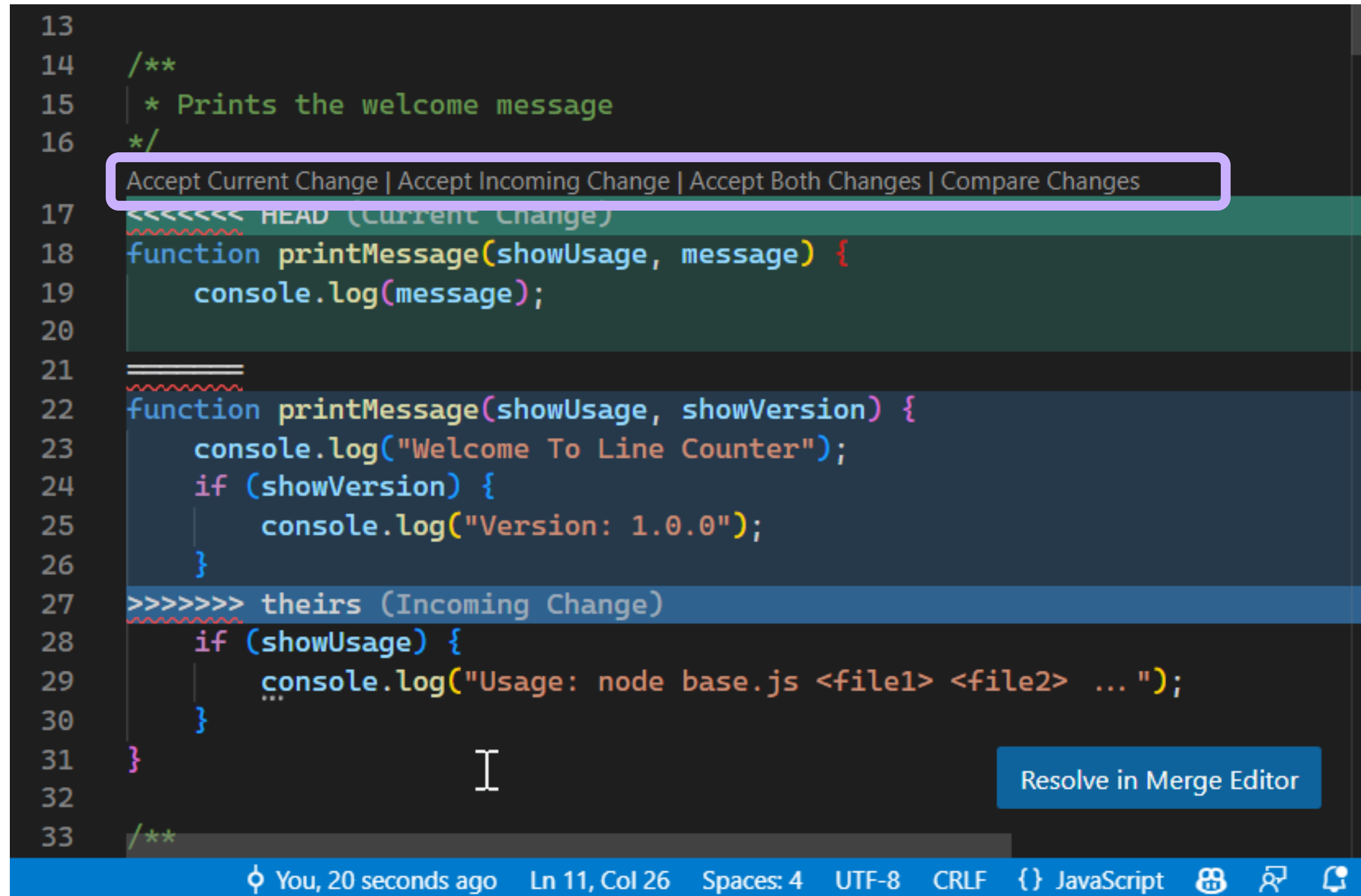
=====

구분선 : 위는 내 변경(HEAD), 아래는
가져온 변경(MERGE_HEAD)을 나눔

>>>>>>> branch-name

충돌 끝 : 병합하려는 상대 브랜치 이름이 표시되며,
여기까지가 충돌 블록임을 표시

VSCode에서의 충돌 액션 (4가지 옵션)



The screenshot shows a VS Code Merge Editor interface with a conflict in a JavaScript file. The editor displays three versions of the `printMessage` function: the current HEAD change (lines 17-20), the incoming change from the merge (lines 22-26), and the theirs change (lines 27-30). A context menu is open over the conflict, showing four options: 'Accept Current Change', 'Accept Incoming Change', 'Accept Both Changes', and 'Compare Changes'. The 'Accept Current Change' option is highlighted. At the bottom right, there is a button labeled 'Resolve in Merge Editor'. The status bar at the bottom indicates the file is 'JavaScript' and shows various icons for file operations.

```
13
14 /**
15  * Prints the welcome message
16  */
17 <<<<<< HEAD (Current Change)
18 function printMessage(showUsage, message) {
19     console.log(message);
20 }
21
22 >>>>>> theirs (Incoming Change)
23 function printMessage(showUsage, showVersion) {
24     console.log("Welcome To Line Counter");
25     if (showVersion) {
26         console.log("Version: 1.0.0");
27     }
28 }
29
30
31 }
32
33 /**
```

Accept Current Change

내 변경(HEAD)만 살리고 가져온 변경은 버립니다.

Accept Incoming Change

가져온 변경(MERGE_HEAD)만 살리고 내 변경은 버립니다.

Accept Both Changes

양쪽 변경을 모두 남깁니다. 순서·중복은 직접 다듬어야 합니다.

Compare Changes

좌우 분할 비교 뷰를 열어 차이를 직접 확인하며 편집합니다.

VSCode에서의 3-way merge editor

JS target.js !

JS Merging: target.js !

merge-git-playground > JS target.js > printMessage

Incoming 7b18bdb • theirs

13

14 /**

15 * Prints the welcome message

16 */

Accept Incoming | Accept Combination (Incoming First) | Ignore

17 function printMessage(showUsage, showVersion) {

18 console.log("Welcome To Line Counter");

19 if (showVersion) {

20 console.log("Version: 1.0.0");

21 }

22 if (showUsage) {

23 console.log("Usage: node base.js <file1>");

24 }

Result merge-git-playground\target.js

Current b71d9b1 • main

13

14 /**

15 * Prints the welcome message

16 */

Accept Current | Accept Combination (Current First) | Ignore

17 function printMessage(showUsage, message) {

18 console.log(message);

19

20 if (showUsage) {

21 console.log("Usage: node base.js <file1>");

22 }

1 Conflict Remaining

13

14 /**

15 * Prints the welcome message

16 */

No Changes Accepted

17 function printMessage(showUsage) {

18 console.log("Welcome To Line Counter");

19

20 if (showUsage) {

21 console.log("Usage: node base.js <file1> <file2> ...");

22 }

Complete Merge

자주 하는 실수

```
VSCode - TERMINAL
$ git switch other-branch
error: Your local changes to the following files
would be overwritten by checkout:
    README.md
Please commit your changes or stash them before
you switch branches.
```

① 커밋하고 전환

```
$ git add .
$ git commit -m "wip: 작업 중"
$ git switch other-branch
```

✓ 정식 커밋으로 기록 (히스토리 남음)

브랜치 전환 전엔 항상 "지금 워킹트리가 깔끔한가?" 확인

→ **git status** (또는 VSCode 좌측 Source Control 패널)

작업한 변경이 아직 **커밋되지 않은 채**로 다른 브랜치로 이동하면 어떻게 될까?

→ 변경사항이 덮어써질 위험이 있으면 Git이 거부함

② 임시 보관(stash)하고 전환

```
$ git stash
$ git switch other-branch
# 다시 돌아왔을 때
$ git stash pop
```

✓ 커밋 없이 임시 보관, 가볍게 처리